Practical DevSecOps Engineering

Production security from assets and authority to restored trust

Birol Tilki

Version 1.0 — 15 August 2026

Contents

How to Use This Book

Who this book is for

This book is for intermediate-to-advanced DevOps, cloud, infrastructure, platform, security, and **SRE (Site Reliability Engineering)** practitioners who need to make security enforceable across a production delivery and operating path.
You should already be comfortable with Linux, Git, containers, **CI/CD (Continuous Integration and Continuous Delivery)**, cloud and Kubernetes fundamentals, infrastructure as code, networking, workload identity, observability, and incident response basics. The book does not teach those subjects from first principles. It uses them as the production surface on which security decisions must operate.

You do not need to be a penetration tester, malware analyst, cryptographer, digital-forensics specialist, privacy lawyer, or compliance auditor. Those disciplines contain important knowledge, but this book has a narrower promise: help you decide what must be protected, model how it can be abused, select proportionate controls, enforce those controls at defensible boundaries, interpret security evidence, and restore trustworthy operation after compromise.

This is not a catalog of security tools. Products and standards change. The durable skill is being able to explain:

- which asset and harm justify a control;
- which trust and authority the control grants;
- what evidence can reveal failure or misuse;
- how an exception remains bounded and temporary; and
- what must be replaced, reconciled, and verified after trust is invalidated.

Relationship to *Practical DevOps Engineering*

This book continues the Northwind Commerce system from *Practical DevOps Engineering*. The earlier book established a production delivery path with fast feedback, immutable artifact identity, provenance, reconciled infrastructure, a bounded Kubernetes runtime, workload identity, observable outcomes, progressive delivery, compatible data changes, safe asynchronous processing, GitOps reconciliation, cost and capacity controls, incident coordination, and reconstruction from durable evidence.
Those capabilities are prerequisites here. This book does not repeat their implementation and present it as new security work. Instead, it asks the adversarial questions that remain:

- Which source, dependency, builder, identity, and policy claims deserve trust?
- Can valid credentials exercise excessive authority?
- Does signed evidence describe an authorized process or merely a cryptographically attributable one?
- Which vulnerabilities are actually exposed, reachable, and capable of harming Northwind?
- Where do secrets and sensitive data exist beyond their intended systems?
- Which preventive failures would Northwind detect independently?
- Can recovery remove attacker capability and persistence rather than merely restore availability?

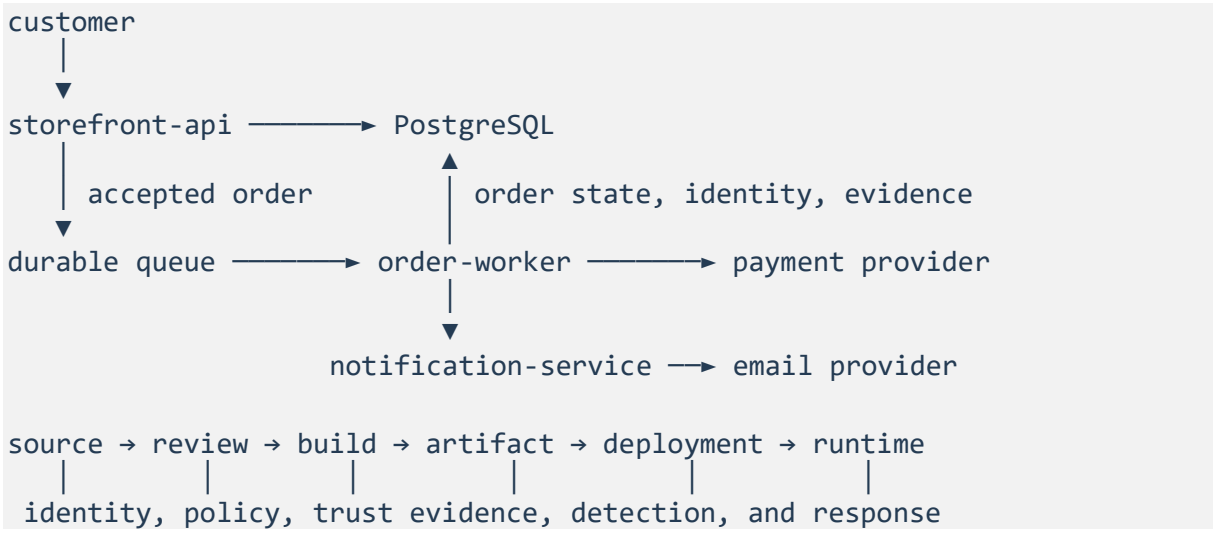
The frozen DevOps manuscript and companion lab remain unchanged. This book has a separate cumulative implementation under:

```
books/labs/devsecops/northwind/
```

The new lab does not read the DevOps lab's working tree at runtime. It carries a small set of reduced interface fixtures whose manifest records the frozen DevOps v1.1 source paths, source checksums, local checksums, and consumers. This makes the prerequisite explicit without copying the whole earlier implementation or allowing later DevOps edits to change this book silently.

The Northwind system

Northwind Commerce remains deliberately small enough to understand and large enough to require real security decisions:



- `storefront-api` serves catalog reads and accepts orders.
- `order-worker` processes accepted orders asynchronously and can cause payment and inventory effects.
- `notification-service` sends non-critical confirmations.
- PostgreSQL stores order and processing state.
- A durable queue separates acceptance from asynchronous completion.
- Payment and email providers are external trust boundaries represented by controlled fixtures.
- The source, build, release, deployment, and runtime path can change what production executes and which authority it receives.

The critical business outcome remains:

> A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

This book adds a critical security outcome:

> Only authorized actors and workloads can cause or observe order-related effects; every trusted release and privileged action is attributable and policy-conformant; suspected compromise is bounded, investigated, eradicated, and followed by evidence that trustworthy operation has been restored.

Security is not separate from the business outcome. An available service that accepts attacker-controlled releases is not healthy. A restored worker that still uses compromised authority is not recovered. A policy-compliant system that exposes unnecessary customer data is not safe merely because its controls generated passing reports.

The cumulative security path

The chapters form one dependency chain:

```
assets, harms, invariants, and ownership
  → threat paths and trust boundaries
  → risk-owned control decisions
  → attributable identity and authorization
  → governed delegation and privilege
  → trusted source and dependencies
  → verifiable build and release admission
  → contextual vulnerability decisions
  → complete secret lifecycle
  → use-based data protection
  → enforceable policy and bounded exceptions
  → runtime confinement
  → correlated security detection
  → investigation and containment
  → eradication and restored trust
  → sustainable operational governance
```

Later chapters consume earlier decisions. The asset register determines what the threat model protects. Threat paths give risk decisions their context. Risk decisions justify identity, supply-chain, secret, data, policy, runtime, and detection controls. Those controls then produce evidence for investigation. Investigation bounds eradication, and recovery evidence becomes the strongest test of the governance claims at the end of the book.

The cumulative attack narrative begins with a compromised maintainer credential used to introduce a malicious dependency and pursue production access. It develops across identity, privilege, source, build, secret, runtime, detection, containment, and recovery chapters.

Not every security failure comes from that attacker. Exercises are labeled according to their relationship with the main narrative:

- **Cumulative attack:** advances the compromised-maintainer intrusion.
- **Connected consequence:** demonstrates harm enabled by that intrusion path.
- **Independent control failure:** proves that the control matters even without the cumulative attacker.

This distinction prevents the main attacker from becoming an unrealistic explanation for every organizational and technical failure.

Four chapter forms

The book does not force every learning objective into the same structure.

Concept-led chapters

A concept-led chapter develops a mental model required for later production decisions. The reader may classify assets, map authority, model trust boundaries, or distinguish forms of harm. Its durable output can be a reviewed reasoning artifact rather than executable code.

Concept-led does not mean detached theory. Every substantial conceptual section must affect a chapter decision, durable artifact, interpretation of evidence, attack diagnosis, or later implementation dependency. Material that affects none of those does not belong.

Decision-led chapters

A decision-led chapter compares approaches under explicit threats and operational constraints. It records the recommendation, trade-offs, residual uncertainty, owner, exception behavior, and review trigger.

A risk score, scanner label, compliance result, or vendor recommendation is an input—not the decision itself.

Implementation-led chapters

An implementation-led chapter starts from a red capability, establishes the necessary mental model, guides a system or policy change, exercises a controlled attack or failure, diagnoses the evidence, and verifies containment or recovery where required.

The chapter tells you what to change, why it matters, how to apply it safely, and what falsifiable evidence proves the claimed result. You are not expected to rediscover essential production steps merely to make the work feel difficult.

Hybrid chapters

A hybrid chapter uses a concept or decision immediately to govern implementation. It still answers one primary production question. Hybrid is not permission to join unrelated chapters for convenience.

There is no theory-to-practice percentage. There is also no artificial exercise added to satisfy one. The practical requirement is consequential reader reasoning, meaningful system change where appropriate, trustworthy evidence, and explicit limits on what has been proved.

Four kinds of evidence

Security mechanisms often produce impressive output while leaving the important claim unanswered. This book separates four evidence categories:

1. **Mechanism evidence** proves that a configured control operated. An admission rule rejected an artifact; an authorization engine denied an action; a detector emitted an alert.
2. **Decision evidence** proves that an owner evaluated the required context and made a bounded decision. A vulnerability treatment records exposure, exploitability, harm, uncertainty, owner, deadline, and review trigger.
3. **Outcome evidence** proves that the protected business or security outcome remains healthy. Orders still reach correct terminal states; unauthorized payment effects do not succeed; expected production behavior remains available after confinement.
4. **Recovery evidence** proves that harmful authority, persistence, and invalid trust were removed or replaced within the declared scope. Old credentials fail, compromised artifacts cannot return, trusted state is reconciled, detections remain active, and business outcomes recover.

These categories are complementary. One cannot silently substitute for another. A passing policy test does not prove the production outcome. A healthy order-success metric does not prove that no sensitive data was exposed. A restarted deployment does not prove that attacker persistence is gone.

Chapter `recover` commands produce chapter-scoped correction or recovery evidence. Only Chapter 15's `verify-recovery` command produces the book's bounded restored-trust claim. Earlier chapters should not treat a local recover target as proof that production trust was restored.

Lab states and commands

Use Python 3.13. From the DevSecOps lab root:

```
python3 -m venv .venv
source .venv/bin/activate
make bootstrap
make test
make lint
make audit
```

The lab uses these states when they support the chapter's learning objective:

- **Start:** cumulative prerequisites exist, but the chapter capability is incomplete.
- **Baseline:** the evaluator runs successfully and proves the declared weakness is present.
- **Complete:** the chapter decision or implementation satisfies independent expectations.
- **Challenge:** a concept or decision scenario exposes incomplete reasoning.
- **Attack:** an inert local input exercises prevention, detection, evidence, or containment.
- **Contained:** modeled active harm and authority are bounded, but trust is not yet assumed restored.
- **Recovered:** required state is reconciled and both security and business recovery evidence pass. Except in Chapter 15, this is chapter-scoped recovery rather than production restored trust.

A successful baseline command means the evaluator correctly found the expected unsafe state. It does not mean the capability is already secure.

Chapter commands follow a stable pattern where applicable:

```
make chapter-NN-baseline
make chapter-NN-checkpoint
make chapter-NN-challenge
make chapter-NN-attack
make chapter-NN-contain
make chapter-NN-recover
make chapter-NN-verify-recovery
```

Not every chapter uses every command. Concept-led and decision-led chapters do not manufacture an attack or recovery sequence when classification, modeling, or decision correction is the real work.

`make chapter-NN-recover` is chapter-scoped. Only `make chapter-15-verify-recovery` produces the bounded restored-trust claim.

Chapter snapshots

Versioned start and complete tags are the exercise snapshots:

```
v1.0-chapter-NN-start
v1.0-chapter-NN-complete
chapter-NN-start
chapter-NN-complete
```

The versioned tags are the immutable release identity. The reader-facing aliases point to the same commits in this freeze and may advance in a later release. Tags are curated exercise snapshots, not merge milestones. Do not infer the teaching contract from Git ancestry.

Attack state is generated from the complete snapshot. A malicious dependency, exposed token, permissive exception, or compromised artifact must never be preserved as if it were a trusted completed reference.

Create a working branch from the start snapshot and compare against the complete reference without merging it as a shortcut:

```
git switch -c my-chapter-NN chapter-NN-start
git diff chapter-NN-start chapter-NN-complete
```

Working-tree procedure

From the DevSecOps lab root:

1. create the Python virtual environment and run `make bootstrap`, `make test`, `make lint`, and `make audit`;
2. start from `chapter-NN-start` (or work in the completed tree if you already have later chapters);
3. run the chapter baseline and confirm that it detects the documented red state;
4. follow the chapter's guided work and preserve its decision or implementation evidence;
5. run the completed checkpoint;
6. run the declared challenge or attack where applicable;
7. verify correction, containment, or recovery where the chapter requires it; and
8. retain the chapter's durable outputs.

A concept-led chapter may change only reviewed YAML or Markdown artifacts. The absence of application code does not make the decision record disposable.

Some commands mutate operational files. Those mutations persist in the working tree. Later chapters depend on earlier generated `build/` outputs and, in Chapters 12 through 16, on specific live register state. If you have already run a later chapter's containment or recovery, restore the incident-ready start with `make lab-reset` rather than assuming a start tag reset the tree.

Files that later chapters may mutate include:

- `identity/subjects.yaml`
- `runtime/contracts/order-worker.yaml`
- `runtime/policies/behavior.yaml`
- `supply-chain/deployment-evidence.yaml`
- `response/case/incident.yaml`
- `policy/enforcement-points.yaml`
- `secrets/inventory.yaml`

Chapters 12–16 command chain

Detection through governance needs generated evidence and live incident state. When those outputs are absent, run this sequence rather than a single later baseline:

```
make chapter-12-attack
make chapter-13-attack
make chapter-14-open
make chapter-14-baseline
make chapter-14-checkpoint
make chapter-14-contain
make chapter-14-recover
make chapter-15-baseline
# … Chapter 15 guided work …
make chapter-15-verify-recovery
make chapter-16-baseline
```

`chapter-14-open` writes incident `INC-2026-0815-01` as `investigating` and sets `compromised-session` to `active`. Complete snapshots keep that incident-ready start. After Chapter 15 verification the live case is `closed` and that session is revoked; `make lab-reset` restores the Chapter 14 start registers.

The complete tag is a reference solution and verification target. It is not a substitute for making and explaining the chapter's decisions, and it should not be merged as a shortcut.

Safe attack simulations

The lab's attacks are deterministic, local, inert, and non-destructive:

- look-alike packages and malicious artifacts are marked text fixtures;
- credential replay uses synthetic identifiers and a modeled authorization engine;
- command-and-control traffic is represented by events and policy decisions rather than real network contact;
- exposed data is synthetic Northwind data with no personal information;
- containment and revocation modify generated lab state only; and
- no real credential, exploit payload, persistence mechanism, malware, or external attack target is included.

These constraints do not weaken the learning objective. The reader must still reason about authority, evidence, attack progression, containment, and recovery. The lab simply avoids confusing a security engineering exercise with permission to attack a real system.

What local verification proves

The local evaluators verify structure, relationships, policy decisions, state transitions, graph reachability, expiry, mutation resistance, evidence integrity, correlation, containment, and reconciliation within the declared model.

They do not prove that a real repository host protects branches, an identity provider validates the same claims, a registry preserves trustworthy provenance, a cloud control plane enforces the modeled authorization, a Kubernetes runtime blocks the same behavior, a telemetry backend retains complete evidence, or a real attacker lacks an unmodeled persistence path.

That final limitation matters most during recovery. The book can prove that inventoried trust roots and modeled persistence paths were replaced or reconciled. It cannot universally prove that no attacker persistence exists. Production confidence depends on the completeness of the inventory, the quality and independence of collected evidence, validation against real systems, and continued heightened monitoring.

Five principles and four security questions

The five principles established for the series remain active:

1. **Blast-radius control:** expose the smallest defensible scope to uncertain change, attack, or failure.
2. **Explicit contracts:** make identity, authority, trust, state, policy, outcomes, and failure behavior reviewable.
3. **Trustworthy evidence:** separate observations and expectations so a mechanism cannot approve itself.
4. **Reconciliation:** compare desired, recorded, external, and actual state, then make disagreement visible and owned.
5. **Recovery:** distinguish corrective action from proof that the protected outcome and required trust are healthy again.

DevSecOps work repeatedly asks four security questions:

1. What asset and harm drive this decision?
2. What trust and authority are granted?
3. What independent detection remains if prevention fails?
4. What evidence proves trust was restored?

Dedicated attacks and failures name only the principles and questions that materially apply. Repeating every label would create ceremony rather than clearer reasoning.

Best Practice and Production Practice

The book retains two labels from the series:

- **Best Practice:** a strong default that is broadly useful.
- **Production Practice:** how that default must be validated or adapted for the actual assets, threats, identities, failure modes, dependencies, evidence quality, operating constraints, and recovery obligations.

For example, short-lived credentials are a strong default. In production, their value depends on verified subject and audience claims, bounded authorization, protected issuance, meaningful expiry, revocation behavior, attributable use, and evidence that replay outside the intended context fails.

How to judge your work

Do not ask only whether the configured control passes. Ask:

> What asset and harm justify this decision, what trust and authority cross the boundary, what evidence could falsify the control's claim, what happens when prevention fails, and how will we prove that trustworthy operation—not merely availability—has been restored?

That question is the reading contract for the chapters ahead.

1. Define What Security Must Protect

Northwind already has controls. Production artifacts are identified by digest. Workloads use bounded identities. Releases can stop when user-visible evidence degrades. Durable state can be reconstructed.

None of those mechanisms answers the first security question:

> What must Northwind protect, from which harms, and who owns the decision?

Without that answer, security work becomes a queue of tool findings and inherited defaults. A scanner reports a vulnerable package. A cloud rule reports a public endpoint. A reviewer asks for encryption. An auditor asks for evidence. Each request may be reasonable, but the team cannot compare them until it knows which business, data, identity, and operational outcomes are at risk.

This chapter establishes that foundation. You will define Northwind's critical assets, name the harms that matter, express security invariants, and assign accountable ownership. The result is not a decorative inventory. Later chapters use it to decide which threat paths deserve attention, which controls are proportionate, which evidence is meaningful, and what recovery must restore.

1. An unsafe security model

The inherited DevOps system protects a critical business outcome:

> A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

That statement is strong operationally. It names success in business terms rather than describing a deployment or process. Security must extend it because an attacker can violate Northwind without making the service unavailable.

Consider four examples:

- An unauthorized actor causes a valid-looking payment for an order the customer never submitted.
- A support user reads complete customer order histories without a legitimate purpose.
- A compromised maintainer admits a malicious release that still returns healthy responses.
- An attacker changes inventory state while every availability indicator remains green.

The DevOps outcome helps expose the damage, but it does not name all protected authority, data, or trust. Northwind needs an explicit security model before it chooses more controls.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run the Chapter 1 baseline:

```
make chapter-01-baseline
```

The command succeeds when it detects the intended unsafe model:

```
chapter 01 baseline: unsafe asset classification correctly detected
```

The fixture calls a payment token a configuration value, assigns no accountable owner, and names no harm. The token may be stored in a configuration system, but storage location does not explain its security significance. It carries authority to cause an external financial effect.

That distinction drives the chapter.

2. The production model: assets, harms, invariants, and ownership

> *Theory — Asset-and-harm model*
>
> This model enables Northwind to select and evaluate controls according to the outcomes they protect rather than the tools that implement them.

An asset is something whose loss or misuse can cause harm

In this book, an asset is something Northwind values because its compromise can cause a meaningful business, customer, security, or operational harm.

An asset can be tangible or intangible:

- a correct terminal order outcome;
- authority to charge a payment method;
- customer and order data;
- authority to admit a production release;
- the integrity of a dependency graph;
- an investigation record whose integrity supports a containment decision; or
- customer trust and the organization's ability to operate within its obligations.

An asset is not limited to data. Identity and authority are often more important than the credential that represents them. A credential is replaceable material. The authority it unlocks can change production, disclose data, or cause a payment.

An asset is also not necessarily a single technical object. The correct terminal state of an order depends on the storefront, queue, worker, database, payment provider, identities, and recovery evidence. Treating only the database row as the asset would miss duplicate external charges and malicious but syntactically valid state transitions.

A resource is where an asset is represented or exercised

A resource is a system object that stores, processes, transmits, or controls access to an asset. PostgreSQL, a queue topic, a repository branch, a cloud role, a container image, and a signing key are resources.

The distinction matters because controls attach to resources while harms attach to assets.

For example:

Resource	Asset represented or exercised	Example harm
PostgreSQL orders table	Correct order state and customer order data	Unauthorized modification or disclosure
Payment provider credential	Payment authority	Unauthorized charge or fraud
Protected production branch	Release authority and reviewed intent	Malicious or unreviewed release
Workload identity token	Dependency-specific workload authority	Unauthorized dependency access
Telemetry store	Security and operational evidence	Investigation based on altered or missing events

If Northwind begins with the resource, it tends to copy generic controls: encrypt the database, rotate the credential, protect the branch, shorten the token, retain the logs. Those can be strong defaults. Starting with the asset and harm reveals what the control must actually accomplish and what evidence could falsify its claim.

Database encryption does not stop an overprivileged application from reading unnecessary fields. Credential rotation does not establish which subject used the old credential. Branch protection does not prove that an approved dependency origin was trustworthy. Log retention does not prove that required events were produced or preserved intact.

Harm is the consequence Northwind is trying to prevent or bound

A harm is a meaningful adverse outcome. It is not the same as a technical event.

“A token appeared in a log” is an observation. The possible harms include unauthorized charge, fraudulent refund, data access, loss of attribution, or use of the credential to enter another trust boundary.

“A container opened a shell” is behavior. The possible harms depend on the workload's authority, accessible data, network reach, deployment context, and the attacker's ability to persist or move laterally.

Naming harm forces useful precision:

- Who or what is affected?
- Which outcome becomes incorrect, unavailable, disclosed, or untrustworthy?
- Can the harm propagate outside the originating resource?
- Can Northwind reverse the effect, or only compensate for it?
- Which evidence would show that the harm occurred?

The familiar confidentiality, integrity, and availability model remains useful:

- **Confidentiality:** information or authority is not disclosed to an unauthorized subject.
- **Integrity:** state, decisions, code, evidence, or effects are not created or changed without authorization.
- **Availability:** required capability and information remain accessible to authorized users at the necessary time.

Northwind also needs business-specific harms such as fraud, duplicate financial effects, loss of attribution, prohibited data use, and inability to establish trustworthy recovery. These are not replacements for confidentiality, integrity, and availability. They make those properties operationally meaningful.

A security invariant states what must remain true

A security invariant is a required or prohibited outcome that should remain true across normal operation, change, attack, failure, containment, and recovery.

Good invariants describe the protected result without prescribing the tool:

- Only an authorized attributable workload may cause a payment effect for an accepted order.
- Order data is disclosed or modified only for an authorized purpose by an attributable subject.
- Every production release is attributable, reviewed, policy-conformant, and bound to trusted evidence.
- A valid order reaches one correct terminal state without duplicate charge or invalid inventory.

Weak statements describe mechanisms or aspirations:

- Use a secrets manager.
- Enable multifactor authentication.
- Scan every image.
- Follow least privilege.
- Keep customer data secure.

The first four may become controls. The last is too vague to evaluate. None states what must remain true when the mechanism fails, is bypassed, produces incomplete evidence, or applies to the wrong resource.

An invariant should be falsifiable. Northwind should be able to identify observations that would contradict it. A payment effect with no accepted order, no authorized workload subject, or no stable operation identity falsifies the payment invariant. A production digest admitted from an unauthorized builder falsifies the release invariant even if its signature is cryptographically valid.

Ownership means decision accountability

Ownership in this chapter does not mean that one team performs every control or responds alone. It means one role or team is accountable for maintaining the asset definition, accepting or escalating relevant risk, reviewing material changes, and ensuring that unclear responsibility does not silently become accepted exposure.

Several groups may participate:

- service owners understand business behavior and application changes;
- security engineers understand threats, control failure, and investigation evidence;
- platform or cloud teams operate shared enforcement mechanisms;
- data owners determine permitted uses and lifecycle obligations;
- incident responders coordinate containment and recovery; and
- executives or risk owners may accept consequences that engineering teams cannot authorize.

Shared work is normal. Shared accountability without a decision owner is dangerous.

An owner must be able to answer:

- Which asset and harm are in scope?
- Which changes require review?
- Who may accept residual risk?
- Where is escalation routed when evidence is missing or contradictory?
- What event triggers reassessment?

An email address alone does not create ownership. A useful ownership record binds a stable role or team identifier to explicit assets and an escalation path.

3. Build Northwind's security foundation

The completed Chapter 1 model uses three files:

```
security-model/assets.yaml
security-model/invariants.yaml
security-model/ownership.yaml
```

The separation is deliberate. Assets describe what is valuable and how its compromise causes harm. Invariants state what must remain true. Ownership identifies who is accountable. Keeping them distinct makes contradictions and missing relationships visible.

> **Practice — Classify assets by harm**
>
> Define what Northwind values before assigning controls to the resources that represent it.

Open `security-model/assets.yaml`. The register begins with four assets:

```yaml
schema_version: 1
kind: asset-register
assets:
  - id: order-outcome
    name: Correct terminal order outcome
    owner: commerce-operations
    harms: [duplicate-charge, invalid-inventory, permanent-order-loss]
  - id: payment-authority
    name: Authority to cause payment effects
    owner: payments-security
    harms: [unauthorized-charge, payment-fraud]
  - id: customer-order-data
    name: Customer and order data
    owner: data-governance
    harms: [unauthorized-disclosure, unauthorized-modification]
  - id: release-authority
    name: Authority to admit production releases
    owner: delivery-security
    harms: [malicious-release, unreviewed-production-change]
```

Notice what is absent. PostgreSQL is not listed as the customer-data asset. A token is not listed as payment authority. Git is not listed as release authority. Those resources will appear in later data, identity, and trust-boundary models. Here, the register remains stable even if Northwind changes database, identity, or repository products.

Inspect each entry and test it with three questions:

1. If this asset were disclosed, modified, unavailable, or misused, could you name a meaningful harm?
2. Would the asset remain important if Northwind replaced the current implementation technology?
3. Is the named owner authorized to maintain the definition and escalate its risk?

Do not expand the inventory into a list of every file, table, queue, and credential. Later chapters need a reviewable set of assets tied to consequential harm, not a configuration-management database disguised as a security model.

Production Practice: Real organizations usually need multiple levels of asset inventory. A high-level harm-oriented register guides threat and risk decisions. More detailed resource inventories support exposure analysis, vulnerability correlation, data lineage, secret rotation, and incident scope. They should reference one another without pretending to serve the same purpose.

> **Practice — State falsifiable security invariants**
>
> Express the outcomes that controls and recovery evidence must preserve.

Open `security-model/invariants.yaml`. The register names four required outcomes:

```
- id: correct-order-terminal-state
  statement: A valid order reaches one correct terminal state without
duplicate charge or invalid inventory.
  assets: [order-outcome, payment-authority]
  harms: [duplicate-charge, invalid-inventory, permanent-order-loss]
  owner: commerce-operations
- id: attributable-payment-effects
  statement: Only an authorized attributable workload may cause a payment
effect for an accepted order.
  assets: [payment-authority, order-outcome]
  harms: [unauthorized-charge, payment-fraud]
  owner: payments-security
- id: governed-order-data
  statement: Order data is disclosed or modified only for an authorized
purpose by an attributable subject.
  assets: [customer-order-data]
  harms: [unauthorized-disclosure, unauthorized-modification]
  owner: data-governance
- id: governed-production-release
  statement: Every production release is attributable, reviewed, policy-
conformant, and bound to trusted evidence.
  assets: [release-authority]
  harms: [malicious-release, unreviewed-production-change]
  owner: delivery-security
```

Each field affects later work:

- `statement` defines the claim.
- `assets` links the claim to protected value.
- `harms` identifies why violation matters.
- `owner` identifies who must resolve ambiguity and review material change.

The invariant intentionally requires both authorization and attribution. A shared credential might successfully authorize a payment while failing to show which workload or operation used it. A perfectly attributable request from an unauthorized subject is still unsafe. Chapter 4 will make this distinction enforceable.

The phrase “for an accepted order” binds authority to business context. Payment access alone is insufficient. Later asynchronous-processing and investigation evidence must connect the external effect to the stable order operation.

Review each invariant. For each one, write down one observation that would falsify it. If you cannot describe contradictory evidence, the statement is probably too vague.

> **Practice — Make ownership reciprocal**
>
> Verify that assets name real owners and that owners explicitly acknowledge accountability for those assets.

Open `security-model/ownership.yaml`:

```yaml
owners:
  - id: payments-security
    accountable_for: [payment-authority]
    escalation: security-incident-commander
```

The asset register points to `payments-security`, and the ownership register points back to `payment-authority`. The checkpoint verifies both directions. This prevents a document from assigning responsibility to a team whose own declared scope does not include the asset.

The `escalation` field is a stable role, not a person's name. The lab verifies that the field exists. A real organization must additionally verify that the role is staffed, reachable, authorized, and rehearsed.

Avoid assigning every asset to “Security.” Security teams may operate controls and guide decisions, but service, business, data, payment, and delivery owners retain knowledge and authority that a centralized security group cannot manufacture.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-01-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 01 checkpoint: asset, harm, invariant, and ownership relationships
verified
```

The audit validates the three Chapter 1 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- all independently required assets exist;
- all independently required invariants exist;
- every asset names at least one harm;
- every asset refers to a known accountable owner;
- each owner acknowledges the assets assigned to it;
- every invariant refers to known assets and an accountable owner; and
- every invariant names at least one threatened harm.

The expected asset and invariant identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that four assets are sufficient for a real commerce company. It does not prove that the harms are complete, the owners have correct organizational authority, or the invariants express every legal and business obligation. Those are judgment claims requiring review with service, business, data, security, and operational stakeholders.

This distinction is the chapter's evidence model:

Evidence category	Chapter 1 evidence
Mechanism evidence	Schemas and the relationship evaluator operated successfully.
Decision evidence	Assets, harms, invariants, owners, and escalation paths are explicit and reviewed.
Outcome evidence	Later controls and production observations can be evaluated against stable security invariants.
Recovery evidence	Not yet produced; later chapters must prove compromised trust and business outcomes were restored.

Chapter 1 primarily creates decision evidence. Pretending that the local checkpoint proves the security outcome would weaken every later chapter.

4. Test the model under failure

Independent control failure — Payment authority hidden as configuration

> **Practice — Diagnose hidden payment authority**
>
> Reclassify an ownerless configuration value by the authority it grants, the harm it can cause, and the evidence recovery requires.

The baseline fixture contains this model:

```yaml
assets:
  - id: payment-token
    name: Payment token configuration value
    owner: unassigned
    harms: []
```

The problem is not merely missing fields. The model classifies the resource and hides the asset.

Severity: high; unauthorized external financial effects and lost attribution.
Plausible harm: unauthorized charges, payment fraud, incorrect order state, dispute and compensation cost, and inability to determine which subject caused an effect.
Potential blast radius: every payment operation permitted by the token's provider-side scope and validity window; a reusable credential may extend beyond one workload or order.
Bounded by: provider-side scope, short validity, per-operation idempotency, transaction limits, workload-specific distribution, anomaly detection, rapid revocation, and reconciliation between accepted orders and provider effects. None repairs the missing asset decision by itself.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control, reconciliation, and recovery.

Security questions

- **Asset and harm:** The asset is authority to cause payment effects; the harms include unauthorized charge and fraud.
- **Trust and authority:** Possession of the token may grant reusable provider authority without a sufficiently attributable workload and order context.
- **Detection after prevention fails:** Northwind needs independently correlated provider, workload, order, and identity evidence capable of revealing effects without valid accepted operations.
- **Evidence of restored trust:** Not yet applicable. Chapter-local correction: the exposed authority must fail, replacement authority must be bounded and attributable, provider effects must reconcile with accepted orders, and legitimate payment outcomes must recover.

Diagnosis

Calling the token “configuration” encourages configuration controls: encrypt it at rest, keep it outside Git, and inject it at runtime. Those may reduce exposure, but they do not answer who can use the authority, for which operations, for how long, with what attribution, or how Northwind detects and recovers from misuse.

The missing harm makes prioritization impossible. The missing owner makes treatment and acceptance ambiguous. The missing invariant leaves no stable claim for later identity, secret, detection, and recovery chapters to enforce.

Correction

The completed model does not elevate the token itself into a crown-jewel asset. It defines `payment-authority` and states the invariant that only an authorized attributable workload may cause a payment effect for an accepted order.

That correction changes later decisions:

- Chapter 4 must bind authorization to an attributable workload subject.
- Chapter 5 must prevent self-approved elevation into payment or production authority.
- Chapter 9 must govern unavoidable payment credentials through issue, use, rotation, exposure, and revocation.
- Chapter 13 must correlate identity, order, provider, and runtime events.
- Chapters 14 and 15 must reconcile payment effects and invalidate compromised authority before declaring restored trust.

The concept is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common modeling errors

Listing every resource as an asset

An exhaustive resource list becomes stale quickly and obscures the few outcomes that should drive risk decisions. Maintain detailed inventories where exposure, lifecycle, and response require them, but connect them to stable harm-oriented assets.

Naming controls as invariants

“All secrets use Vault” binds the security claim to one product and says nothing about authorized use, exposure, revocation, or recovery. State the required outcome first; select the mechanism later.

Assigning ownership without authority

A team cannot own a risk decision if it lacks authority to change the system, fund treatment, accept residual impact, or escalate to someone who can. Record the real decision path.

Treating confidentiality as the only security property

For Northwind, malicious but valid-looking changes to order, release, identity, and payment state can be more damaging than disclosure. Integrity, attribution, purpose, and recoverability deserve explicit treatment.

Using high-level assets without downstream traceability

“Customer trust” may be valuable, but it is too broad to govern an authorization rule by itself. Connect it to specific data, authority, service, and evidence outcomes that later controls can evaluate.

Assuming absence of evidence means absence of harm

If Northwind cannot correlate a payment effect with an accepted operation and attributable subject, it has missing evidence—not proof that the effect was authorized. Chapter 13 will make telemetry completeness an explicit detection concern.

6. What changed

Before	After
Northwind listed technical resources without naming the protected outcome.	**Assets describe stable business, data, identity, and operational value.**
A payment token looked like ordinary configuration.	**Payment authority is modeled through unauthorized-charge and fraud harms.**
Security goals described tools or broad aspirations.	**Falsifiable invariants state what must remain true across operation, attack, and recovery.**
Asset ownership was implied by system operation.	**Assets and owners acknowledge accountability in both directions and name escalation paths.**
A valid schema could appear to prove a sound decision.	**Structural, decision, outcome, and recovery evidence remain explicitly distinct.**
Findings and controls had no stable business reference.	**Later threats, risks, controls, detections, and recovery decisions inherit the same asset-and-harm model.**

What changed was not merely three YAML files. Northwind now has a falsifiable security contract that later chapters can threaten, enforce, observe, and restore.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Asset-and-harm and ownership registers	security-model/assets.yaml and security-model/ownership.yaml	They retain the stable protected outcomes, plausible harms, accountable owners, and escalation paths that later risk decisions reference.
Security invariant register	security-model/invariants.yaml	It retains the falsifiable claims that later controls, attacks, evidence, and recovery must preserve or restore.

These artifacts should change when Northwind's business use, authority, data, dependencies, or plausible harms materially change—not whenever a tool produces a new finding.

What You Learned

Assets are valued outcomes, data, or authority whose compromise can cause harm. Resources store, transmit, represent, or exercise those assets but are not automatically the assets themselves. Technical events become security-relevant through their plausible consequences. Security invariants state falsifiable required or prohibited outcomes without prescribing tools, while accountable owners provide real decision and escalation paths.

Schema and relationship checks can prove structural completeness within declared scope. They cannot prove that human judgment is correct. A concept earns its place when it changes later production decisions, evidence, diagnosis, or recovery.

Prove It

> **Independent Practice — Model refund authority**
>
> Decide how Northwind protects refund authority without copying the payment-authority model mechanically.

Northwind plans to add customer-service refunds. A refund changes an external financial state, but its actor, business context, limits, approval path, and harm differ from an order-worker payment.

Extend the Chapter 1 model without adding implementation policy yet:

1. Decide whether refund authority is a new asset or a bounded use of `payment-authority`.
2. Name at least two plausible harms, including one that does not require stolen credentials.
3. Write a falsifiable invariant connecting subject, authorization, order context, amount or scope, and attribution.
4. Assign an accountable owner and escalation role.
5. Identify one observation that would falsify the invariant.

6. Explain which material change would trigger review of your decision.

Do not copy the payment entry and rename it. A support user's refund authority has different delegation, fraud, approval, evidence, and recovery consequences. Your durable output is the decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 1 capability when you can explain why payment authority matters more than the token that represents it, trace every invariant to known assets and harms, describe evidence that would falsify each invariant, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now knows what it must protect and who owns the definitions. It still does not know where trust is granted, which actors can cross those boundaries, or how a compromised maintainer can progress from source authority toward production and payment harm.

Chapter 2 turns the asset and invariant registers into a threat model with explicit data and authority flows, trust boundaries, attack prerequisites, control assumptions, and prioritized abuse paths.

2. Model Threats Across Trust Boundaries

Northwind now knows which outcomes, authority, and data it must protect. It has explicit harms, security invariants, and accountable owners. That foundation still does not explain how a plausible actor can violate an invariant.

A connectivity diagram shows that `order-worker` calls the payment provider. A threat model asks harder questions: what authority crosses that boundary, which claims the provider validates, which assumptions remain outside the validation, and how a compromised maintainer could arrive at the same boundary through source, build, release, and runtime transitions.

This chapter turns Northwind's asset model into explicit data and authority flows, trust boundaries, and prioritized attack paths. It does not attempt to enumerate every imaginable attack. It creates a reviewable model of how plausible capabilities can reach the harms defined in Chapter 1.

1. A threat list without a system model

A weak threat model often begins like this:

```
actor: hacker
capability: attack-system
action: compromise-everything
threatens: [be-secure]
```

The words sound security-related, but none supports a production decision. The actor capability is undefined. The prerequisite says nothing about existing access. The action does not follow a real Northwind flow. The threatened goal is not a known invariant. No owner can determine whether the path is plausible, interrupted, detectable, or accepted.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-02-baseline
```

The baseline succeeds when it rejects the generic path:

```
chapter 02 baseline: generic, untraceable attack path correctly rejected
```

A list of threats can prompt discussion. It becomes an engineering model only when its claims connect to the system, trust transitions, protected invariants, controls, missing evidence, and ownership.

2. The production model: flows, boundaries, and attack paths

> *Theory — Trust transitions and attack paths*
>
> This model enables Northwind to identify where untrusted claims gain authority and where prevention or detection can interrupt plausible harm.

Threats describe possible harm, not inevitable events

A threat is a circumstance or actor capability that could violate a security invariant. An attack path describes how that capability can progress through the modeled system.

Keep these terms separate:

Term	Meaning	Northwind example
Threat actor	A person, group, or process capable of acting against an invariant	External attacker controlling a maintainer session
Capability	What the actor can presently do	Act through an authenticated maintainer session
Prerequisite	A condition required before a step is plausible	Dependency changes are permitted through the session
Action	What the actor attempts at one step	Introduce a look-alike payment dependency
Attack path	Connected actions across real flows and boundaries	Maintainer → source → build → registry → production → provider
Harm	The adverse outcome if an invariant is violated	Unauthorized charge or malicious release

An actor label alone is rarely useful. “Nation state,” “insider,” and “hacker” may imply different motivation, patience, and resources, but Northwind still needs testable capabilities and prerequisites. A junior employee with valid production access may have more immediate authority than a sophisticated external actor with no foothold.

Threat modeling does not predict the future. It records what the current system makes plausible, which assumptions support that judgment, and what evidence could change it.

Data flow and authority flow are different

Data-flow diagrams show what moves. Security decisions also need to show what the receiver permits the sender to cause.

The `order-payment-effect` flow carries an order identifier, payment request, and stable operation identifier. It also exercises `cause-payment` authority. The data may be syntactically valid while the effect is unauthorized.

Likewise, the source-to-build flow carries reviewed source and a dependency lock. It exercises authority to trigger a build. The build-to-registry flow carries an artifact and evidence while exercising publication authority.

Flow question	Data view	Authority view
What crosses?	Source, artifact, token, order, event, or claim	Permission to build, publish, deploy, read, modify, or cause an external effect
What can be malformed?	Content, encoding, schema, identity value	Scope, subject, approval, context, duration, or delegated power
What is the consequence?	Incorrect or exposed information	Unauthorized state transition or external effect

Modeling only data misses confused-deputy failures: a trusted workload can receive attacker-influenced input and use its own legitimate authority to cause harm.

A trust boundary is where claims require validation

A trust boundary is not simply a network segment. It is a transition where the receiving side must decide whether to accept claims, data, identity, intent, or authority from a different trust context.

Northwind's boundaries include:

- a human session proposing and approving source;
- reviewed source entering a build execution;
- a builder publishing an artifact;
- registry evidence entering production admission;
- a deployment controller changing a runtime workload;
- a workload requesting a payment effect; and
- a support session requesting protected order data.

Each boundary record names:

- the source and destination trust zones;
- what the receiver validates;
- what the design still assumes; and
- who owns the boundary.

Validation and assumption must not be conflated. The payment provider can validate workload subject, audience, operation identity, and order context while still assuming its idempotency behavior is correct and its evidence remains available. Later verification must either test those assumptions or keep the uncertainty visible.

Controls interrupt paths; evidence tests the interruption

A control can prevent one step, reduce its impact, expose it to detection, or help recover afterward. It does not erase the attack path from the model.

Northwind already promotes artifacts by digest and uses workload identity. Those controls narrow ambiguity and authority. They do not prove that a dependency origin was approved, a valid maintainer session was uncompromised, or an admitted workload behaved as intended at runtime.

Keep paths that remain plausible under control failure. Record both existing controls and missing evidence. Later chapters will turn those gaps into owned risk and control decisions.

Mnemonic catalogs such as **STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)** can help teams remember those categories. They are prompts, not proof of coverage. STRIDE threat categories. A long catalog with no connection to Northwind's flows and invariants is weaker than a smaller set of explicit paths with real prerequisites and owners.

3. Build Northwind's threat model

The Chapter 2 model uses three files:

```
threat-model/system.yaml
threat-model/boundaries.yaml
threat-model/attack-paths.yaml
```

> **Practice — Map data and authority flows**
>
> Describe what crosses each production connection and what the receiver allows the sender to cause.

Open `threat-model/system.yaml`. One flow is:

```yaml
- id: order-payment-effect
  from: order-worker
  to: payment-provider
  carries: [order-id, payment-request, operation-id]
  authority: [cause-payment]
  boundary: workload-to-provider
```

The stable component and flow identifiers matter. Attack paths reference them rather than copying prose that can drift independently.

Inspect every flow and ask:

1. What data or claim crosses?
2. Which authority is exercised?
3. Which security invariant can be affected?
4. Does the flow cross a decision boundary?
5. Which observations would later show accepted and rejected use?

Do not infer trust from location. Two processes in one cluster can have different identities and authority. A managed external service can enforce stronger claims than an internal shared account. The boundary follows the decision, not the network drawing.

> **Practice — Separate validation from assumption**
>
> Record what each receiver verifies and what remains an explicit uncertainty owned by the system.

Open `threat-model/boundaries.yaml` and inspect production admission:

```
- id: registry-to-production
  from_zone: artifact-storage
  to_zone: deployment-control
  validates: [artifact-digest, source-identity, builder-identity,
admission-policy]
  assumptions: [attestation-verifier-independent, policy-source-protected]
  owner: delivery-security
```

The boundary does not say “the registry is trusted.” It states which claims admission validates and which properties it assumes. Chapter 7 will make the validation enforceable. Chapter 13 will preserve evidence capable of revealing failures that prevention misses.

Review assumptions as future work, not hidden truth. An assumption can be accepted temporarily, converted into a verified control, monitored through detection, or rejected by redesigning the boundary.

> **Practice — Trace the cumulative attack path**
>
> Connect the compromised maintainer's capability to real flows, boundaries, threatened invariants, controls, and missing evidence.

Open `threat-model/attack-paths.yaml`. The cumulative path begins with control of an active maintainer session and progresses through six named steps. Its final step attempts a payment effect outside valid order context.

The path threatens three Chapter 1 invariants:

- `governed-production-release`;
- `attributable-payment-effects`; and
- `correct-order-terminal-state`.

It records existing controls—reviewed source, digest promotion, workload identity, and provider idempotency—without declaring the path impossible. It also records missing evidence about session assurance, dependency origin, and correlated runtime behavior.

The second path, `overprivileged-support-data-access`, models a valid support session with excessive field access. It threatens `governed-order-data`. This is not part of the cumulative attacker story. It proves that authorized subjects can violate purpose and data boundaries without stolen credentials.

Prove the capability

Run the audit and Chapter 2 checkpoint:

```
make audit
make chapter-02-checkpoint
```

Expected output includes:

```
artifact validation: passed
chapter 02 checkpoint: flows, boundaries, attack paths, and invariants
verified
```

The checkpoint proves within the declared model that:

- flows reference known components and boundaries;
- all independently required boundaries exist;
- attack steps reference real flows and agree with their boundaries;
- paths state prerequisites and missing evidence;
- threatened invariants exist in Chapter 1; and
- every independently required priority invariant has a modeled attack path.

It cannot discover undocumented architecture, prove assumptions true, establish objective likelihood, or predict a real attacker's behavior.

4. Test the model under failure

The cumulative compromised-maintainer path is the completed model in section 3. This dedicated failure is independent: it proves the model rejects untraceable stories even when no Northwind attacker is present.

Independent control failure — A generic attacker bypasses every real boundary

> **Practice — Reject an untraceable threat story**
>
> Diagnose why dramatic attacker language cannot support a control or evidence decision without system traceability.

Severity: decision failure; a generic model can hide high-impact paths while creating false confidence.
Plausible harm: misdirected controls, missed release or payment abuse, unowned evidence gaps, and delayed containment.
Potential blast radius: every asset whose protection relies on the incomplete threat model.
Bounded by: independently required invariants, stable system identifiers, boundary ownership, cross-reference checks, and periodic review after material change.
Primary principles: explicit contracts, trustworthy evidence, reconciliation, and blast-radius control.

Security questions

- **Asset and harm:** The model must terminate at known assets and invariants rather than “security” in general.
- **Trust and authority:** Every step must identify the real boundary and authority being crossed.
- **Detection after prevention fails:** Missing evidence must be explicit enough to become a detection hypothesis later.
- **Evidence of restored trust:** Not yet applicable; this chapter establishes the path that later containment and recovery must close.

Diagnosis

The unsafe fixture refers to an unknown flow, a fictional `trusted-network` boundary, and an undefined `be-secure` invariant. The checkpoint rejects it because no reviewer can determine where the actor entered, which validator failed, which owner acts, or what observation could falsify the story.

Correction

Correct the model by selecting a real actor capability, stating prerequisites, tracing named flows and boundaries, linking Chapter 1 invariants, recording existing controls without assuming perfection, naming missing evidence, and assigning an owner.

5. Production reality

Best Practice: threat-model material changes to assets, identities, trust boundaries, dependencies, and authority before release.

Production Practice: choose a review cadence and participation model that match Northwind's change rate and harm. The service owner, delivery owner, data owner, payment owner, and security practitioner see different parts of the same path.

Avoid these common failures:

- treating a network diagram as a threat model;
- listing attacker types without capabilities or prerequisites;
- trusting authenticated input without checking authorization and purpose;
- deleting paths because one preventive control exists;
- modeling every imaginable attack at equal depth;
- hiding uncertainty inside words such as “trusted,” “internal,” or “secure”; and
- allowing the document to drift after identities, dependencies, providers, or release authority change.

A useful model is incomplete but explicit. Its limitations and assumptions are review inputs. An exhaustive-looking model with stale or implicit boundaries is harder to challenge and therefore more dangerous.

6. What changed

Before	After
The architecture showed connectivity.	**Flows state both transferred data and exercised authority.**
“Internal” and “trusted” hid validation decisions.	**Boundaries name validated claims, assumptions, and owners.**
Threats were generic attacker labels.	**Paths state capabilities, prerequisites, actions, and real transitions.**
Controls made paths disappear from discussion.	**Existing controls and missing evidence remain visible on the path.**
Security goals were detached from Chapter 1.	**Every priority path terminates at known invariants and harms.**
Review completeness was an opinion.	**Independent expectations and semantic checks reject missing coverage and broken references.**

Northwind now has an attack model that later risk decisions can prioritize and later controls can interrupt, observe, and recover from.

Durable outputs

Artifact	Location	Keep it because
System and trust-boundary model	threat-model/system.yaml and threat-model/boundaries.yaml	They record data, authority, validation, assumptions, and ownership at every modeled transition.
Prioritized attack-path register	threat-model/attack-paths.yaml	It connects actor capabilities to real flows, controls, missing evidence, and Chapter 1 invariants.

What You Learned

A threat model is a decision model, not a catalog of frightening possibilities. Data flows show what crosses a connection; authority flows show what the receiver allows the sender to cause. Trust boundaries identify where claims require validation and where assumptions remain. Attack paths connect plausible capabilities and prerequisites to real transitions, protected invariants, existing controls, missing evidence, and owners.

Prove It

> **Independent Practice — Model compromised support access**
>
> Extend the support-data path without treating a valid identity as proof of authorized purpose.

Assume an attacker controls an active support session but cannot change source or production workloads. Model a path toward unauthorized customer-order disclosure. State the capability, prerequisites, flow, boundary, validated claims, remaining assumptions, threatened invariant, existing controls, missing evidence, and owner. Then explain which prevention and detection decisions Chapter 3 should evaluate.

Next

Northwind now knows what it protects and how plausible capabilities can cross trust boundaries toward harm. It still cannot decide which paths require immediate treatment, which uncertainty is acceptable, or how to select complementary controls without turning severity labels into automatic decisions.

Chapter 3 turns the threat model into owned, evidence-backed risk and control decisions with explicit residual uncertainty and review triggers.

3. Turn Risk into Owned Control Decisions

Northwind has explicit assets, harms, invariants, trust boundaries, and attack paths. It can explain how a compromised maintainer might introduce a malicious dependency and progress toward production payment authority. It also models a valid support session accessing data beyond its purpose.

Those paths do not decide what Northwind should do first. A threat model describes plausible routes to harm. Risk work compares their context, uncertainty, impact, and urgency, then assigns an accountable treatment. Without that step, severity labels and tool queues quietly become decision makers.

1. A critical label without a risk decision

A weak record says:

```
id: scanner-critical
severity: critical
treatment: fix-now
```

It does not identify a deployed asset, attack path, exposure, reachability, business harm, uncertainty, owner, residual risk, or review trigger. “Critical” may justify immediate investigation. It cannot complete the decision.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-03-baseline
```

The baseline succeeds when the evaluator rejects the score-only record:

```
chapter 03 baseline: score-only risk decision correctly rejected
```

2. The production model: risk is owned uncertainty about harm

> *Theory — Contextual risk and control portfolios*
>
> This model enables Northwind to select proportionate treatment without mistaking numeric precision, severity, or compliance status for a production decision.

Likelihood and impact are judgments supported by evidence

Risk concerns uncertainty about whether a threat will cause harm and how serious that harm would be. Northwind uses qualitative labels—low, medium, high, and critical—because the available evidence does not support precise probabilities or universal financial values.

Dimension	Question	Relevant evidence
Exposure	How can the actor reach the prerequisite or boundary?	Network path, identity access, deployment state, feature use
Likelihood	How plausible is progression under current conditions?	Known exploitation, control strength, frequency, attacker capability
Impact	Which asset and harm can be affected, and how far can it spread?	Payment scope, data sensitivity, business outcome, reversibility
Uncertainty	What important fact is missing, stale, or assumed?	Incomplete telemetry, unknown reachability, untested provider behavior

A qualitative label must not hide reasoning. Two teams can assign “medium” for different reasons. The record must preserve those reasons so later evidence can challenge them.

Inherent and residual risk answer different questions

Inherent risk considers the path before the selected control portfolio. Residual risk describes what remains after existing and planned controls operate as expected.

Residual risk is never automatically zero. Independent approval can reduce the chance of one compromised maintainer approving a sensitive change. It does not eliminate collusion, reviewer error, identity compromise, build misuse, or runtime behavior that prevention did not anticipate.

Treatment is a decision, not a status

Northwind can:

- **Mitigate** by reducing likelihood or impact;
- **Avoid** by removing the risky activity or authority;
- **Transfer** defined consequences through another party or contract without transferring accountability for every harm;
- **Accept** residual risk through an authorized, evidenced, time-aware decision; or
- **Monitor** uncertainty while preserving triggers that force reassessment.

“Open,” “in progress,” and “backlog” are workflow states, not treatments.

Controls should cover failure, not merely prevention

A priority path needs a portfolio when no single mechanism can make its assumptions true:

Control role	Purpose on the maintainer path
Prevent	Require independent approval and admit dependencies only from governed origins.
Detect	Correlate unusual source, identity, admission, and runtime behavior.
Respond	Revoke affected authority and isolate the path before harm expands.
Recover	Rebuild affected artifacts and workloads from trusted roots and reconcile outcomes.

Defense in depth is not the number of tools. Controls are complementary when their failure modes and evidence are sufficiently independent.

3. Make the risk and control decisions

> **Practice — Record contextual risk**
>
> Tie each treatment to a real attack path, affected assets, exposure, likelihood, impact, uncertainty, owner, and review trigger.

Open `risk/risk-register.yaml`. The cumulative path is recorded as externally exposed, medium likelihood, and critical impact. More important than those labels are the preserved uncertainties:

```
uncertainty:
  - maintainer-session-compromise-rate
  - dependency-review-effectiveness
  - runtime-detection-coverage
treatment: mitigate
residual_risk: >-
  A valid session may still influence reviewed source; independent
admission
  and runtime detection must bound progression.
```

The record does not convert unknowns into invented decimals. It makes them reviewable and gives later identity, supply-chain, and detection chapters a reason to produce better evidence.

> **Practice — Select complementary controls**
>
> Choose controls according to where they interrupt, reveal, contain, or recover the attack path.

Open `risk/control-decisions.yaml`. The maintainer decision includes prevention, detection, response, and recovery. Its rationale states why one identity or source control is insufficient. It also names expected evidence, an owner, a due date, and a material-change trigger.

A control without expected evidence cannot be evaluated. A due date without an owner is a reminder, not accountability. A mitigation without residual risk implies unjustified certainty.

> **Practice — Compare risk rather than compare labels**
>
> Explain why an exposed high-impact authorization path can outrank a critical but unreachable finding.

The support-data path has high impact and internal exposure. Its valid identity makes misuse plausible even without stolen credentials. A scanner finding labeled critical but absent from deployed code may require verification and monitoring, while the active support boundary requires treatment now.

This does not mean unreachable equals harmless. Reachability evidence can be wrong or change. Record the uncertainty and review trigger instead of deleting the finding.

Prove the capability

Run:

```
make audit
make chapter-03-checkpoint
```

The checkpoint verifies known attack paths and assets, explicit uncertainty and residual risk, owned mitigation decisions, and a complete control-role portfolio for the priority cumulative path. It cannot prove that the likelihood label is objectively correct or that the selected controls will work in a real provider.

4. Test the decision under failure

Independent control failure — Severity displaces exposure and harm

> **Practice — Correct a score-driven priority inversion**
>
> Reorder two findings using deployed context, exploitability, harm, uncertainty, and ownership rather than severity alone.

Severity: high decision risk; resources can be diverted from an exposed harmful path.
Plausible harm: delayed authorization repair, unauthorized data access, unmanaged residual risk, and false confidence from shrinking critical-count dashboards.
Potential blast radius: every asset governed by the distorted remediation queue.
Bounded by: attack-path traceability, exposure and asset context, authorized ownership, review triggers, and independent evidence.
Primary principles: explicit contracts, trustworthy evidence, reconciliation, and blast-radius control.

Security questions

- **Asset and harm:** Priority follows the affected asset and plausible harm, not the scanner's label alone.
- **Trust and authority:** The exposed support role already holds usable authority; the unreachable component may not execute in production.
- **Detection after prevention fails:** Accepted or deferred risks need monitoring tied to the condition that justified the decision.
- **Evidence of restored trust:** Not yet applicable; later chapters must implement and test the selected controls.

Diagnosis

The score-only record cannot distinguish theoretical component presence from an active authorization path. It also hides uncertainty and leaves no owner able to revise the decision.

Correction

Verify deployment and reachability, preserve uncertainty, prioritize the exposed path according to harm, assign both records an explicit treatment and owner, and define triggers that reopen either decision when evidence changes.

5. Production reality

Best Practice: use consistent risk criteria and require authorized ownership for treatment and acceptance.

Production Practice: calibrate labels against Northwind's assets, operating constraints, fraud exposure, data use, and recovery capability. Do not compare labels across teams until their definitions and evidence expectations are comparable.

Avoid multiplying likelihood and impact numbers merely to produce a ranked decimal. Arithmetic can conceal weak assumptions. Preserve decision context, contradictory evidence, due dates, exceptions, and material-change triggers.

6. What changed

Before	After
Severity selected work automatically.	**Attack path, exposure, harm, and uncertainty support an owned priority decision.**
Unknown facts disappeared inside a score.	**Uncertainty is explicit and becomes a later evidence requirement.**
Mitigation meant adding one preventive tool.	**Complementary prevention, detection, response, and recovery cover different failure modes.**
“Backlog” acted as a treatment.	**Mitigate, avoid, transfer, accept, or monitor are explicit decisions.**
Passing controls implied no remaining risk.	**Residual risk and review triggers remain visible.**

Durable outputs

Artifact	Location	Keep it because
Owned risk register	risk/risk-register.yaml	It preserves attack context, assets, exposure, uncertainty, treatment, residual risk, ownership, and review triggers.
Control decision register	risk/control-decisions.yaml	It records the complementary portfolio, rationale, evidence, deadlines, and owners that later chapters must implement.

What You Learned

Risk is owned uncertainty about harm, not a synonym for vulnerability severity. Qualitative labels support decisions only when their context and uncertainty remain visible. Treatments must be explicit, residual risk must survive control selection, and priority paths often require complementary preventive, detective, responsive, and recovery controls.

Prove It

> **Independent Practice — Decide under incomplete exploitability evidence**
>
> Choose a treatment without converting missing evidence into false numeric certainty.

Northwind finds a high-severity library in `notification-service`. Runtime reachability is unknown, the service has no payment authority, and exploitation could expose confirmation metadata. Compare it with the active support-data path. Record both decisions, their uncertainty, owners, evidence needs, residual risk, and review triggers.

Next

Northwind has selected owned controls for its priority paths. The first implementation problem is identity: shared accounts, reusable automation credentials, and accumulated roles can make a valid action neither sufficiently attributable nor appropriately authorized.

Chapter 4 makes human and automation access attributable and bounded.

4. Make Human and Automation Access Attributable

Northwind has selected controls for the compromised-maintainer path. The first implementation problem is identity. A valid credential can still represent the wrong subject, target the wrong audience, live too long, or exercise authority accumulated for another purpose.

This chapter makes human and automation actions attributable and authorization decisions bounded by subject, action, resource, environment, issuer, audience, lifetime, and current status.

1. Valid credentials with ambiguous authority

Northwind begins with shared access patterns, reusable automation tokens, and inherited roles. A maintainer credential can propose source changes, but accumulated production authority makes the same session capable of attempting deployment. Automation tokens survive beyond one run and can be replayed outside their intended system.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-04-baseline
```

The baseline succeeds when it reproduces the unsafe start state:

```
chapter 04 baseline: shared subject allowed unattributable production
authority with a reusable day-long token
```

One shared subject, `northwind-ci`, presents a day-long reusable token to production and is allowed to reconcile the deployment. The decision also carries no policy version, so nothing in the record explains which rules produced it or which human or workflow acted. Both halves matter: the authority is too broad, and the evidence cannot survive later investigation.

2. The production model: identity claims authorize context, not possession

> *Theory — Attributable identity and bounded authorization*
>
> This model enables Northwind to distinguish a verified subject from the actions that subject may perform in one resource, environment, audience, and time boundary.

Authentication and authorization answer different questions

Authentication establishes which subject a verifier believes is present and why. Authorization decides whether that subject may perform one action on one resource under the current context.

Session assurance is the confidence attached to that authentication event: how the subject authenticated, whether stronger factors or device checks were required, how recently authentication occurred, and whether the session satisfies the sensitivity of the requested action. It is contextual evidence, not a permanent property of the user.

Decision	Question	Required context
Authentication	Which subject presented this claim, and which issuer vouches for it?	Subject, issuer, session assurance, expiry
Authorization	May this subject perform this action here and now?	Action, resource, environment, role or attributes, policy version
Attribution	Can later evidence connect the action to the subject and decision?	Stable subject, claim identity, request context, result, reason

Authentication success must not imply authorization. A maintainer may be authentic and still have no production deployment authority.

Human, automation, and workload identities need distinct subjects

A shared account destroys attribution. A human, release workflow, deployment controller, and runtime workload have different lifecycles, evidence, and authority. They must not collapse into “CI” or “service account.”

Automation should exchange short-lived federated proof for target-specific access. The token audience identifies the intended receiver. Lifetime bounds replay. The subject identifies the workflow or controller. Policy restricts the permitted action and resource.

Issuer trust is not subject authorization

An issuer can mint tokens for several audiences. Trusting `northwind-oidc` for both registry and production does not mean every subject from that issuer belongs in both systems.

Northwind therefore binds audiences twice:

- the trust policy states which audiences an issuer may assert; and
- the subject register states which of those audiences belong to that subject.

This prevents a release workflow from presenting a cryptographically valid production-audience token when its purpose is artifact publication.

3. Implement the identity and authorization contract

> **Practice — Register attributable subjects**
>
> Replace shared identities with stable human and automation subjects, explicit issuers, audiences, roles, and revocation state.

Replace the shared `northwind-ci` entry in `identity/subjects.yaml` with subjects that have their own issuer, audience, role, and status:

```
subjects:
  - {id: maintainer-alice, type: human, issuer: northwind-human-idp,
audiences: [northwind-source], roles: [source-maintainer], status: active}
  - {id: release-workflow, type: automation, issuer: northwind-oidc,
audiences: [northwind-registry], roles: [artifact-publisher], status:
active}
  - {id: deployment-controller, type: automation, issuer: northwind-oidc,
audiences: [northwind-production], roles: [production-reconciler], status:
active}
  - {id: compromised-session, type: human, issuer: northwind-human-idp,
audiences: [northwind-source], roles: [source-maintainer], status: active}
```

`compromised-session` is the modeled stolen maintainer session used by later attack, detection, and containment evidence. It remains `active` in the operational register through Chapter 13. Chapter 4 containment writes a generated revocation snapshot; Chapter 14 is the first command that revokes this subject in `identity/subjects.yaml`. Keeping a stable identifier lets denial, replay, and incident evidence stay attributable rather than collapsing into an unknown subject.

> **Practice — Bound roles by action, resource, and environment**
>
> Make authority narrow enough that a valid identity cannot silently inherit production actions.

Split the accumulated `maintainer` role in `identity/roles.yaml` into one role per authority. `source-maintainer` permits `propose-change` on `northwind-source` in the repository environment:

```
policy_version: "2026-08-15.1"
roles:
  - id: source-maintainer
    allows:
      - {action: propose-change, resource: northwind-source, environment:
repository}
```

It does not permit artifact publication or production reconciliation. Separate automation subjects own those actions. The `policy_version` is not decoration: every decision trace records it, so a later investigation can tell whether an allowed action was permitted by the rules in force at the time or by rules that have since changed.

> **Practice — Validate issuer, audience, lifetime, and replay behavior**
>
> Reject claims that are valid-looking but intended for another receiver, too long-lived, reusable, or revoked.

Bound sessions and replay in `identity/trust-policy.yaml` with `max_session_seconds: 3600` and `reusable_tokens_allowed: false`, then narrow each issuer to the audiences it may assert. The evaluator now checks subject status, issuer, issuer audience, subject audience, maximum session lifetime, reusable-token policy, role, action, resource, and environment. Its result retains denial reasons instead of returning an unexplained Boolean.

> **Practice — Emit a decision trace that survives the decision**
>
> Make every allow and deny reconstructable without access to the running evaluator.

Each decision appends one line to `identity/access-events.jsonl` binding subject, action, resource, environment, issuer, audience, session lifetime, reusable claim, both policy versions, result, and reasons. Abridged to the fields that carry the denial:

```
{"subject": "release-workflow", "action": "publish-artifact", "resource":
"northwind-registry",
 "audience": "northwind-production", "result": "deny", "reasons":
["audience-rejected"]}
```

An allow with no trace and a deny with no reason are equally unusable later. Chapter 13 correlates these events with dependency, elevation, and runtime signals, so the fields it needs must exist at the moment of the decision, not be reconstructed afterward.

Prove the capability

```
make audit
make chapter-04-checkpoint
```

The checkpoint proves that a maintainer can propose source, the release workflow can publish to the registry, and only the deployment controller can reconcile production. It also proves the two denials that matter most — a maintainer session attempting production reconciliation, and the release workflow presenting a production audience — and it fails if any emitted trace is missing a field. It does not exercise a real identity provider or cloud authorization service.

4. Test the design under failure

Cumulative attack — Reuse maintainer identity as production authority

> **Practice — Deny and recover from compromised identity use**
>
> Exercise the reusable-token and inherited-role attempt, revoke its subject, and prove legitimate automation remains functional.

Severity: high; successful misuse could change the production workload.
Plausible harm: malicious release, loss of attribution, unauthorized production state, and progression toward payment authority.
Potential blast radius: Northwind production resources reachable by the inherited role.
Bounded by: subject-specific audiences, short lifetime, non-reusable proof, narrow roles, explicit denial evidence, and revocation.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Release authority and order outcomes are exposed to malicious production change.
- **Trust and authority:** A valid maintainer session has source authority only; it must not inherit registry or production authority.
- **Detection after prevention fails:** Denial events preserve subject, attempted action, resource, audience, and reason.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: the compromised subject fails after generated revocation while legitimate release automation still publishes with the correct audience.

Diagnosis

Run `make chapter-04-attack`. The modeled session is still active, but it is reusable, too long-lived, aimed at the wrong audience, and lacks the production role. Four independent controls deny it before revocation. Read the last line of `identity/access-events.jsonl`: the denial carries a stable claim identifier, subject, attempted action and resource, presented audience, session lifetime, and policy versions. That is what makes the event usable as detection input instead of noise.

Run `make test`. The Chapter 4 mutation tests relax one control at a time — issuer audiences, subject audiences, revocation, session lifetime, reusable tokens, and role scope — and require that the same production attempt still fails. With every control relaxed the attempt is allowed, which is what keeps the test honest: it proves each control is independently load-bearing rather than proving the denial once and calling it depth.

Those mutations exposed a subtler defect. Issuer-level audience trust originally allowed the release workflow to present a production audience, because `northwind-oidc` legitimately serves both the registry and production. Binding audience to the subject as well closed that gap, and the checkpoint now holds the boundary as an explicit deny case.

Containment
Run `make chapter-04-contain`. This performs the containment transition in generated state: it copies the register, revokes `compromised-session` in that copy, denies a claim that would otherwise permit source access specifically because of revocation, and writes `build/chapter-04-contained.yaml`. It does not mutate `identity/subjects.yaml`. Preserve the claim identifier and denial reasons in `identity/access-events.jsonl` for Chapter 13 correlation. Operational revocation of this session waits for Chapter 14.

Recovery
Run `make chapter-04-recover`. Recovery passes only when the old session remains denied and legitimate registry automation remains allowed. Revocation that breaks every workflow is containment without operational recovery.

5. Production reality

Best Practice: use individual human identities and short-lived federated automation identities with least authority.

Production Practice: validate issuer configuration, subject mapping, audience semantics, session assurance, policy propagation, revocation delay, clock behavior, and audit completeness in the actual systems. **OIDC (OpenID Connect)** requires a client to reject an ID token whose audience does not include that client; a cryptographically valid token for another audience is not authorization. OpenID Connect Core.

Role names are not evidence of narrow authority. Expand them into concrete actions and resources. Review role accumulation, dormant subjects, issuer changes, and machine identities that no owner can explain.

6. What changed

Before	After
Shared identities obscured who acted.	**Human and automation subjects are distinct and attributable.**
Authentication implied authorization.	**Every decision binds subject, action, resource, environment, and policy context.**
Issuer trust applied to every subject audience.	**Issuer and subject audience constraints must both pass.**
Reusable tokens survived workflow execution.	**Short-lived, non-reusable proof is required.**
Maintainer roles accumulated production authority.	**Source, publication, and reconciliation authority belong to separate subjects.**
Decisions left no record of their own context.	**Every allow and deny emits a trace carrying claims, policy versions, and reasons.**
Revocation success could break all automation.	**Recovery proves compromised access fails while legitimate automation works.**

Durable outputs

Artifact	Location	Keep it because
Identity and trust inventory	identity/subjects.yaml and identity/trust-policy.yaml	They retain subject, issuer, audience, lifetime, replay, and revocation boundaries.
Authorization matrix	identity/roles.yaml	It makes permitted actions, resources, environments, policy version, and separation of authority reviewable.

Decision traces accumulate in `identity/access-events.jsonl`. That file is generated evidence rather than a reviewed artifact, so it stays out of version control, but its field contract is what Chapter 13 builds detections on.

What You Learned

Authentication establishes a subject claim; authorization decides one contextual action; attribution preserves who attempted what and why it was allowed or denied. Issuer trust, subject audience, lifetime, role, resource, and revocation must all agree. Recovery proves compromised authority stays invalid without disabling legitimate automation.

Prove It

> **Independent Practice — Add read-only production diagnosis**
>
> Design a short-lived human diagnostic role without turning observation into deployment authority.

Specify the subject, issuer, audience, session assurance, permitted observations, prohibited changes, expiry, revocation, and audit evidence. Explain why reusing the deployment-controller role would destroy attribution and separation of authority.

Next

Normal identities are now attributable and bounded. Exceptional delegation, temporary elevation, impersonation, and break-glass access can still create an unowned bypass.

Chapter 5 governs privileged operations and delegation.

5. Govern Delegation and Privileged Operations

Chapter 4 bounded normal identity and authorization. Production still needs exceptional access: responders isolate workloads, operators revoke identities, support engineers diagnose sensitive failures, and automation sometimes acts on delegated authority. If those paths become permanent, self-approved, or unaudited, they recreate the privilege Chapter 4 removed.

1. Emergency access without a contract

A compromised maintainer claims an emergency, approves their own elevation, requests production reconciliation for a day, and records a review before the session has even ended. The word “emergency” does not make the authority legitimate, and a completed field does not prove that a lifecycle occurred.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-05-baseline
```

The baseline reads the unsafe request fixture rather than synthesizing an attack in memory. It succeeds when self-approval, excessive duration, an out-of-scope action, and impossible lifecycle ordering are rejected independently.

2. The production model: privilege is bounded delegation

> *Theory — Delegation, elevation, and break glass*
>
> This model enables Northwind to grant exceptional authority without creating a permanent or unowned bypass.

Delegation allows one authorized subject to confer a bounded capability on another. Elevation temporarily increases a subject's authority. Impersonation acts as another subject and therefore demands especially strong purpose and evidence. A support engineer may temporarily elevate or impersonate here, but Chapter 10 decides which customer fields that subject may access for a declared support purpose. Break glass is a predesigned emergency path, not permission to ignore identity and policy.

Element	Required decision
Requester and subject	Who asks, and who will exercise the authority?
Approver	Which independent authority may approve it?
Purpose	Which declared production condition justifies access?
Scope	Which action, resource, and environment are permitted?
Duration	When does authority expire automatically?
Evidence	Which request, decision, use, and revocation events remain?
Review	Who determines whether use was justified and controls should change?

Just-in-time access limits duration. Just-enough administration limits scope. Neither is sufficient without attribution, independent approval, revocation, and review.

3. Define the privileged workflow

> **Practice — Bound privileged authority**
>
> Define the maximum duration, allowed emergency actions, approval separation, and required evidence.

Open `privilege/policy.yaml`. Northwind permits only `isolate-workload` and `revoke-subject` through this break-glass path, for at most 30 minutes. Production reconciliation is deliberately excluded.

> **Practice — Record an independently approved request**
>
> Bind requester, subject, approver, purpose, action, resource, duration, and incident ticket.

The approved fixture gives `responder-alice` permission to isolate `order-worker` for 15 minutes under `SEC-1042`. The incident commander approves; the requester cannot approve themselves. `self_approval_allowed: false` is evaluated policy, not documentation.

> **Practice — Close the delegation lifecycle**
>
> Expire or revoke access and complete after-action review while the decision evidence remains attributable.

Open `privilege/requests/approved.yaml` and follow its timestamps: requested, approved, issued, ended, then reviewed. `privilege/sessions.jsonl` separately preserves requested, approved, used, revoked, and reviewed events. The evaluator rejects an impossible sequence and enforces the policy's maximum duration and review deadline.

The review is not ceremony. It asks whether the emergency path was necessary, whether scope was excessive, whether ordinary controls failed, and whether policy or automation should change. Dual control means the requester cannot authorize their own exceptional authority. For emergency changes to production, this break-glass contract may grant narrow containment actions, but the production change still travels through the separately governed release path.

Prove the capability

Run:

```
make audit chapter-05-checkpoint
```

The audit validates the request against `schemas/privilege-request.schema.json`. The checkpoint verifies required fields, independent approval, bounded duration and action, valid chronology, and timely review. It models the workflow locally; it does not grant real production privilege.

4. Test the decision under failure

Cumulative attack — Invent an emergency and self-approve elevation

> **Practice — Reject privileged self-escalation**
>
> Exercise an invented emergency that requests excessive production authority without independent review.

Severity: critical—the request seeks self-approved production authority outside the emergency allowlist.
Plausible harm: malicious deployment, concealment of source compromise, and progression toward payment authority.
Potential blast radius: production resources reachable by the requested reconciliation authority.
Bounded by: action allowlist, independent approval, short expiry, individual identity, revocation, and review.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, and recovery.

Security questions

- **Asset and harm:** Release authority and production integrity drive the decision.
- **Trust and authority:** Emergency declaration does not authorize self-approved reconciliation.
- **Detection after prevention fails:** The denied request preserves requester, purpose, scope, duration, and missing review.
- **Evidence of restored trust:** Not yet applicable. Chapter-local correction: no elevation was issued; the initiating identity remains subject to Chapter 4 revocation and investigation.

Diagnosis

Run `make chapter-05-attack`. The request fails self-approval, duration, action-scope, and lifecycle-order requirements independently. The attack remains denied even if any one of the other controls were misconfigured.

Containment

Run `make chapter-05-contain`. It proves that the denied authority was never issued and has no entry in session-use evidence. The initiating identity remains subject to the revocation and investigation path established in Chapter 4.

Recovery

Run `make chapter-05-recover`. Recovery proves more than denial: a legitimate responder can still complete the requested, independently approved, used, revoked, and reviewed lifecycle. Use a real incident identifier, an authorized independent approver, the narrow containment action, a short duration, automatic expiry, attributable session evidence, and after-action review.

5. Production reality

Best Practice: prefer individual, just-in-time, just-enough privilege with automatic expiry.

Production Practice: test approval availability, identity-provider failure, revocation delay, evidence retention, and the ability to use break glass during the very outage it is meant to address. A path that cannot work under failure will be bypassed; a path that never expires will become normal access.

6. What changed

Before	After
Emergency access implied broad bypass.	**Break glass permits only declared containment actions.**
Requesters could approve themselves.	**Independent approval separates request from authorization.**
Privilege persisted until someone remembered it.	**Duration and revocation close authority automatically.**
Purpose was free-form justification.	**Purpose, ticket, action, resource, and subject form a reviewable contract.**
Use ended without learning.	**After-action review can redesign the failed normal path.**

Durable outputs

Artifact	Location	Keep it because
Privileged-access decision contract	privilege/policy.yaml and privilege/requests/approved.yaml	They retain approval separation, purpose, scope, chronology, duration, and review.
Privileged session evidence	privilege/sessions.jsonl	It distinguishes request, approval, use, revocation, and review as attributable events.
Privilege-request schema	schemas/privilege-request.schema.json	It prevents incomplete lifecycle records from being accepted as evidence.
Break-glass mini-runbook	privilege/break-glass-runbook.md	It preserves the emergency sequence, revocation, and after-action obligations.

What You Learned

Privileged access is a bounded delegation lifecycle, not a special role assigned forever. Requester, subject, approver, purpose, scope, duration, evidence, revocation, and review must agree. Break glass remains narrow and attributable even when normal operations fail.

Prove It

> **Independent Practice — Design provider emergency access**
>
> Define a temporary path for reconciling payment-provider state without granting general payment or deployment authority.

Specify the approver, subject, permitted observations and actions, transaction bounds, duration, evidence, revocation, and after-action review.

Next

Identity and privilege paths are bounded. Attacker-controlled source and dependency inputs can still enter the build under valid maintainer authority. Chapter 6 establishes trust in source and dependencies.

6. Establish Trust in Source and Dependencies

Chapter 5 bounded exceptional authority. A valid maintainer identity can still propose unsafe code, and an ordinary dependency update can still resolve to attacker-controlled content. Northwind must decide which inputs may enter a build without treating authentication, review, or a familiar package name as proof of trust.

1. A reviewed change with an untrusted result

The compromised maintainer changes the payment dependency from `northwind-payment` to the plausible-looking `northwind-payments`. Resolution selects a high version from a public registry. The same maintainer supplies the only approval.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-06-baseline
```

The start-state policies trust the broad `northwind-` public namespace, treat the compromised maintainer as the payment-path owner, and lock the look-alike package. The command succeeds because it proves that this permissive configuration admits the unsafe input:

```
chapter 06 baseline: permissive source and dependency policy admitted
unsafe input
```

The weakness is not a malfunctioning verifier. Its expectations grant the wrong trust.

2. The production model: trust the resolved input, not its label

> *Theory — Source authenticity and dependency provenance*
>
> This model enables Northwind to reconstruct what entered the build and why policy permitted it.

Source authenticity answers where a revision came from and which protected reference contains it. Review independence asks whether someone other than the change author authorized a sensitive change. Neither claim proves that the resulting dependency bytes came from the intended publisher.

Dependency identity is a relationship among name, version, registry, namespace, and content hash. Omitting any part creates ambiguity:

Evidence	Question answered	What it cannot prove alone
Origin and revision	Which repository state was evaluated?	That its changes were independently authorized
Protected reference	Which governed line of development contains it?	That branch protection was correctly configured in the live host
Author and approvers	Who proposed and reviewed the change?	That the resolved package is the intended content
Registry and namespace	Which distribution authority supplied the name?	That the selected version and bytes match review
Locked version and hash	Which exact content resolution must reproduce?	That the original package or publisher was trustworthy
Resolution decision	Why the complete input set passed policy	That the later builder preserved those inputs

Dependency confusion exploits an authority mismatch: an internal-looking name resolves from a registry whose namespace Northwind does not govern. The class was demonstrated when public packages reused internal names and were selected by version comparison. Dependency confusion. Typosquatting exploits human recognition: a nearby spelling looks legitimate. Version pinning does not solve either problem if the wrong package was pinned. A hash protects integrity after selection; it does not make a malicious selection trustworthy.

The inverse mistake is equally dangerous. A trusted registry is not permission to consume every package it hosts. Registry approval and namespace approval are separate decisions.

3. Enforce source and dependency admission

> **Practice — Govern source authority**
>
> Declare trusted origins, protected references, sensitive paths, independent review, and bounded update automation.

Open `supply-chain/source-policy.yaml`. Changes under `services/payment/` and `supply-chain/` need an approver distinct from the author. `dependency-bot` is the attributable automation subject permitted to prepare routine updates; it cannot approve its own change.

Ownership and approval are related but different. `supply-chain/ownership.yaml` identifies who must care about a path. The source policy determines which review relationship is sufficient to admit a particular change. A broad repository approval must not silently replace sensitive-path ownership.

> **Practice — Bind package identity to its distribution authority**
>
> Approve registries and namespaces together, then require exact locked versions and hashes.

Open `supply-chain/dependency-policy.yaml` and `supply-chain/lock.yaml`. The private registry may supply `northwind-payment`; the public registry may supply only the separately declared public namespace. The lock record binds the approved package to version `3.4.1` and one content hash. It also binds the public transitive dependency `northwind-public-http-signing` to its registry, version, and hash.

Vendoring can reduce registry availability risk and preserve reviewed content locally, but it transfers patch intake, licensing review, storage, and provenance responsibility to Northwind. Automated updates reduce staleness, but their identity, allowed manifests, approval rules, and evidence must remain bounded. Neither technique removes the trust decision.

> **Practice — Reconstruct the resolved graph independently**
>
> Compare observed origin, revision, attributable update claim, path-owner approvals, direct and transitive resolution, registry, version, and hash with policy and lock expectations.

`supply-chain/resolution-evidence.yaml` is the observation presented to the verifier. The policy and lock are separate expectations. Run:

```
make audit chapter-06-checkpoint
```

The audit validates all five governed supply-chain artifacts. The checkpoint passes only when a sensitive change has approval from an independent path owner, its update identity has an attributable claim, and every direct and transitive dependency matches its allowed registry namespace and locked content. This local resolver model does not query or secure a hosted repository or registry; production must obtain equivalent facts from those systems and protect the policy source independently.

4. Test the design under failure

Cumulative attack — Admit a look-alike payment package

> **Practice — Separate a valid identity from a trustworthy input**
>
> Exercise the inert resolution fixture containing a look-alike package, public-registry origin, wildcard request, and self-review.

Severity: critical; the change targets code that can influence payment effects.
Plausible harm: attacker-controlled build input, fraudulent payment behavior, sensitive-data exposure, or a later production foothold.
Potential blast radius: builds and environments that consume the altered payment dependency.
Bounded by: protected references, sensitive-path ownership, independent approval, registry and namespace policy, exact locks, and hash verification.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Payment authority, release authority, and correct order outcomes drive the decision.
- **Trust and authority:** Source approval grants no implicit authority to introduce a package from an unapproved distribution namespace.
- **Detection after prevention fails:** The admission decision retains the attempted name, registry, version, hash, author, claim, approvers, revision, result, and individual denial reasons for later correlation.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: the unsafe revision remains quarantined, the approved graph is reconstructed, and bounded update automation can still produce an admissible change.

Diagnosis

Run `make chapter-06-attack`. The verifier rejects missing independent review, missing path-owner approval, the unapproved namespace, and the unknown package independently. Each reason names the affected path or dependency. The command writes `build/chapter-06-attack-decision.json`; a familiar name and high version do not satisfy any trust contract.

Containment

Run `make chapter-06-contain`. It consumes the denied attack decision and writes `build/chapter-06-quarantine.json`. The unsafe revision is now explicitly quarantined rather than assumed different from the approved revision. Record the denial through the Chapter 4 identity path and retain the decision for Chapter 13 detection work. The operational `compromised-session` register remains active until Chapter 14 containment.

Containment stops further admission; it does not prove that earlier builds were clean. Production responders must identify every build and environment that could have consumed the suspect revision.

Recovery

Run `make chapter-06-recover`. Recovery requires the quarantine record, reconstructs the approved direct and transitive graph, verifies the policy's attributable-update requirement, and writes a separate allow decision for bounded automation. If the unsafe input had entered a build, recovery would also invalidate affected artifacts and rebuild them through the Chapter 7 trust chain.

5. Production reality

Best Practice: evaluate source identity, independent authorization, registry namespace, exact resolution, and content integrity as one admission decision.

Production Practice: enforce protected references in the Git host, restrict package-manager registry fallback, reserve internal namespaces where possible, authenticate private registries, retain immutable resolution evidence, and alert on policy denials. Test mirror outages and update-bot failure. A control that requires developers to bypass it during ordinary registry failure is not a durable control. GitHub, for example, documents that protected branches can require reviews and restrict who can push. About protected branches.

Lockfile changes deserve the same review as the manifest that caused them. Reviewing only the human-readable request while ignoring resolved transitive changes leaves the actual build input unaudited. Conversely, a large mechanical lock change needs tooling that explains origin, version, hash, and ownership without asking reviewers to infer meaning from thousands of lines.

6. What changed

Before	After
A valid maintainer could introduce any plausible package name.	**Sensitive changes require independent authorization.**
Registry choice was a resolver default.	**Registry and namespace authority are explicit policy.**
A version string represented dependency identity.	**Name, registry, version, and hash identify resolved content.**
Review covered the manifest but not necessarily its resolution.	**Resolution evidence reconstructs the complete admitted input.**
Blocking one change was treated as success.	**Containment preserves the graph; recovery proves trusted updates still work.**

Durable outputs

Artifact	Location	Keep it because
Source trust policy	supply-chain/source-policy.yaml and supply-chain/ownership.yaml	They preserve origin, protected-reference, ownership, approval, and automation authority.
Dependency admission policy	supply-chain/dependency-policy.yaml and supply-chain/lock.yaml	They bind registries and namespaces to exact resolved content.
Resolution evidence	supply-chain/resolution-evidence.yaml	It records the independently reviewable input admitted to the next build.
Admission and quarantine evidence	build/chapter-06-attack-decision.json, build/chapter-06-quarantine.json, and build/chapter-06-recovery-decision.json	They preserve denial, containment, and restored admission as distinct state transitions.

What You Learned

Authentication tells Northwind who proposed a change; it does not make the change trustworthy. Source origin, protected references, sensitive-path ownership, independent review, registry namespace, locked resolution, and content hashes form complementary controls. The verifier must reconcile observations against expectations that the resolution mechanism cannot rewrite to approve itself.

Prove It

> **Independent Practice — Govern a public dependency migration**
>
> Decide how Northwind may replace a private payment client with a public package without trusting its name or popularity.

Specify publisher and registry authority, ownership, independent approvers, namespace, exact version and hash, transitive resolution evidence, rollback conditions, and the observations that would invalidate the decision.

Next

Northwind now admits only governed source and dependency inputs. Chapter 7 ensures that bounded builders, attestations, release approval, and deployment admission preserve that trust from source to running artifact.

7. Enforce a Verifiable Build and Release Chain

Chapter 6 established which source and dependency inputs Northwind may trust. That decision can still be lost inside the build. A signed artifact may come from an unauthorized builder, use different inputs, omit required evidence, or target an environment its approval never covered.

1. A valid signature on an untrusted build

The compromised maintainer obtains build-token authority and submits an artifact produced by `compromised-builder-v2`. Its signature is cryptographically valid. Its provenance also says the builder was neither isolated nor hermetic.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-07-baseline
```

The permissive start policy trusts the compromised builder and asks only whether the signature is valid. The baseline therefore exposes the weakness without pretending a successful denial already exists:

```
chapter 07 baseline: signature-only policy admitted an untrusted non-
isolated builder
```

2. The production model: attestations make claims; policy grants trust

> *Theory — Provenance, roots of trust, and admission*
>
> This model enables Northwind to decide whether one artifact may cross one release boundary.

An attestation is a signed statement about a subject. Provenance describes how an artifact was produced. A signature can establish that a particular key signed those claims and that they have not changed. It cannot establish that the signer, builder, inputs, parameters, or target are authorized. The **SLSA (Supply-chain Levels for Software Artifacts)** build requirements distinguish merely existing provenance from authentic provenance and from an isolated build platform. Apply that distinction here: a valid signature can authenticate claims without proving the builder was isolated or authorized.

Claim	Admission question	Failure hidden by signature-only checks
Source revision	Does it equal the admitted Chapter 6 revision?	The builder used different source
Dependency decision	Does it bind the admitted resolution?	Resolution changed after review
Builder identity	Is this builder currently trusted?	A valid key was used by an unauthorized builder
Isolation and hermeticity	Could undeclared state influence the result?	Host or network inputs entered the build
Build parameters	Were only reviewed release parameters used?	Debug or unsafe options changed the artifact
Artifact digest	Which immutable output is being admitted?	Evidence refers to a different image
SBOM (Software Bill of Materials) and transparency	Are required evidence and a durable record present?	Claims disappear or dependencies cannot be inspected
Release approval and target	Who authorized this digest for this environment?	Staging approval is replayed for production

The root of trust is the set of identities, keys, policy sources, and verification mechanisms whose correctness the decision depends on. Adding signatures expands the evidence graph; it does not eliminate roots. Trust rotation is therefore an operational capability: Northwind must be able to revoke a builder or key, replace it, rebuild, and prove old authority no longer admits releases.

Hermeticity means declared inputs determine the build without undeclared network or host dependencies. Isolation bounds interference between builds and limits what a compromised build can affect. These properties overlap, but neither implies the other.

3. Enforce the complete release chain

> **Practice — Bound builders and build parameters**
>
> Declare trusted builders, signing keys, isolation, hermeticity, and allowed release parameters.

Open `supply-chain/build-policy.yaml`. `northwind-builder-v3` is the trusted builder. Both the older and current signing keys are initially recognizable so Chapter 7 can distinguish a cryptographically valid signature from an authorized builder. The pre-rotation artifact uses `release-key-v2`; containment revokes that key, and recovery must produce a different digest with `release-key-v3`.

> **Practice — Bind provenance to the admitted inputs**
>
> Require provenance to name the Chapter 6 source revision and dependency-resolution decision.

Open `supply-chain/provenance.yaml`. Its source claim binds revision `8a19e60`, while its dependency claim binds the content-addressed Chapter 6 resolution decision. A dependency graph change therefore invalidates the binding even if someone reuses the same source revision. The artifact has an immutable digest, and the attestation names builder identity, build properties, parameters, signing key, SBOM digest, and transparency entry.

The verifier compares these observations with independent policies and Chapter 6 evidence. It does not accept a builder's statement that the builder itself is trusted. This follows the verification shape described by the SLSA artifact verification guidance: validate the subject and digest, then evaluate provenance fields against trusted expectations.

> **Practice — Admit one digest to one environment**
>
> Evaluate independent release approval, evidence requirements, policy version, and target environment before deployment.

Open `supply-chain/admission-policy.yaml` and `supply-chain/deployment-evidence.yaml`, then run:

```
make audit
make chapter-07-checkpoint
```

The audit validates the governed artifacts. The checkpoint admits the chain only when source, dependencies, builder, parameters, signature, key, evidence, approval, artifact, and target agree. This lab uses fixture attestations; production must verify signatures, repository decisions, transparency inclusion, builder isolation, and deployed digests against their authoritative systems.

4. Test the design under failure

Cumulative attack — Present a signed artifact from an untrusted builder

> **Practice — Reject cryptographic validity without policy trust**
>
> Evaluate a correctly signed artifact produced after build-token misuse by a non-isolated, non-hermetic builder.

Severity: critical; the artifact seeks production release authority.
Plausible harm: malicious production code, falsified order effects, credential exposure, or persistence through a trusted-looking release.
Potential blast radius: production workloads and business effects reached by the artifact.
Bounded by: admitted inputs, trusted builder identity, isolation, hermeticity, parameter policy, release approval, transparency, and digest-bound deployment.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Release authority, payment authority, and correct order outcomes drive admission.
- **Trust and authority:** A valid signature authenticates a claim; only policy authorizes the builder, key, artifact, and production target.
- **Detection after prevention fails:** The admission decision retains builder, signing key, source revision, artifact digest, target, policy version, result, and individual denial reasons.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: compromised signing trust is revoked, a trusted builder produces a new digest with the rotated key, and deployment evidence names that exact digest and policy.

Diagnosis

Run `make chapter-07-attack`. The artifact is denied because its builder is untrusted, non-isolated, and non-hermetic even though its signature is valid. The command writes `build/chapter-07-attack-decision.json`.

Containment

Run `make chapter-07-contain`. It derives the compromised builder and signing key from the denial decision, then writes them to `build/chapter-07-revocations.json`. Every ordinary admission evaluation automatically consumes active revocations; callers cannot accidentally admit an old key by omitting an optional argument. Rejection stops this deployment, but responders must still locate earlier artifacts produced with that authority.

Recovery

Run `make chapter-07-recover`. Recovery requires the revocation record and first proves that the legitimate pre-rotation artifact signed by `release-key-v2` is no longer admissible. It then evaluates a rebuilt artifact with a new digest from `northwind-builder-v3` using `release-key-v3`. Finally, it reconciles the digest, environment, admission result, and policy version with production deployment evidence and writes a distinct recovery decision.

Service availability alone would not prove recovery. The deployed digest, source and dependency decisions, builder identity, rotated key, policy version, and production target must form one consistent chain.

5. Production reality

Best Practice: admit an artifact only by verifying the semantics of its complete evidence chain against independently governed trust policy.

Production Practice: isolate ephemeral builders, minimize network access, protect signing operations from build steps, retain transparency records, version admission policy, and reconcile the deployed digest after admission. Test key rotation, builder revocation, verifier outage, log unavailability, and rollback. Decide explicitly which failures deny production admission and which require a bounded emergency process.

Higher provenance levels can improve consistency, but labels do not replace examination of the claims Northwind needs. A sophisticated attestation that omits target approval or binds the wrong dependency resolution remains insufficient.

6. What changed

Before	After
A valid signature was treated as sufficient trust.	**Signature validity and policy authorization are separate checks.**
Builder identity was descriptive metadata.	**Only current, bounded builder identities may produce releasable artifacts.**
Provenance could describe inputs without binding admission.	**Source and dependency decisions must match the artifact chain.**
Release approval was detached from an environment.	**Approval binds one digest to one target under one policy version.**
Deployment success implied recovery.	**Revocation, rebuild, rotation, and deployed-digest reconciliation prove restored trust.**

Durable outputs

Artifact	Location	Keep it because
Release trust policy	supply-chain/build-policy.yaml and supply-chain/admission-policy.yaml	It preserves builders, keys, build properties, approval, evidence, and allowed targets.
Admission evidence record	supply-chain/provenance.yaml, supply-chain/deployment-evidence.yaml, and generated build/chapter-07-*.json	Reviewed fixtures define the trusted chain; gitignored generated decisions demonstrate denial, revocation, and recovery for the exercise run.

What You Learned

Signing proves integrity and signer possession, not authorization or truth. Trustworthy release admission verifies source, dependencies, builder, isolation, parameters, artifact, evidence, approval, and environment as one chain. Recovery removes compromised roots, rebuilds from admitted inputs, rotates affected trust, and reconciles the exact deployed digest.

Prove It

> **Independent Practice — Rotate a trusted builder without stopping releases**
>
> Design an overlap window in which Northwind replaces a builder and signing key without letting the retired trust remain valid indefinitely.

Specify old and new identities, admissible overlap, transparency evidence, revocation trigger, rebuild scope, rollback constraints, deployed-digest verification, and the observation that closes the migration.

Next

Northwind can now verify its source-to-production trust chain. Chapter 8 turns vulnerability findings about that chain and its runtime components into prioritized, owned treatment decisions.

8. Prioritize Vulnerabilities by Exploitability and Harm

Chapter 7 established a trustworthy release chain. Its **SBOM (Software Bill of Materials)**, scanners, and runtime inventory now produce findings faster than Northwind can remediate them. A severity label identifies technical potential; it does not decide what threatens production first.

1. When critical displaces urgent

Northwind's queue puts a critical flaw in `legacy-pdf` above a high-severity flaw in `order-parser`. The critical library is deployed, but its vulnerable export path is disabled and unreachable. The high flaw is internet reachable, enabled on the order path, and known to be exploited.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-08-baseline

chapter 08 baseline: severity-only queue displaced the exploitable order-path flaw
```

The baseline does not prove the critical finding is harmless. It proves that severity alone cannot establish the remediation order.

2. The production model: findings inform risk decisions

> *Theory — Severity, exploitability, exposure, and reachability*
>
> This model enables Northwind to turn inconsistent scanner output into owned treatment decisions.

A weakness is a defect or undesirable property. A vulnerability is a weakness that can be exercised under relevant conditions to violate a security property. A finding is a tool's claim that such a condition may exist. Those are not interchangeable.

Context	Decision contribution	Dangerous shortcut
Affected component and version	Identifies the candidate vulnerable state	Package name alone proves exposure
Deployed configuration	Shows whether the affected behavior exists	Present in an SBOM means exploitable
Reachability	Shows whether an execution or data path can exercise it	Unreachable means permanently harmless
Exposure	Identifies who or what can reach the path	Internal means trusted
Known exploitation	Raises urgency with observed attacker behavior	No public exploit means no plausible attack

Context	Decision contribution	Dangerous shortcut
Asset and harm	Connects technical effect to Northwind's outcome	Critical score implies critical business harm
Compensating control	May bound likelihood or blast radius	A control eliminates the need for review
Uncertainty	Exposes what the decision does not yet know	Missing evidence should be treated as safety

Reachability is time-bound evidence about a particular version, configuration, and path. It may change when a feature is enabled, an import appears, or deployment topology changes. Therefore an unreachable finding can be monitored or deferred only with evidence, an owner, review triggers, and an expiry—not dismissed.

Severity and exploitability answer different questions. Severity estimates technical consequence under assumed exploitation. The **CVSS (Common Vulnerability Scoring System)** specification states that Base scores measure intrinsic severity and should not be used alone to assess risk. CVSS v4.0 specification. Exploitability considers prerequisites and attacker feasibility. Exposure and reachability connect those assumptions to Northwind's deployed state. Harm determines why the organization cares.

3. Make context-backed treatment decisions

> **Practice — Normalize and correlate findings**
>
> Deduplicate raw scanner claims by vulnerability, component, and affected version while preserving their sources and derived confidence.

Open `vulnerabilities/findings/raw.yaml` and `vulnerabilities/findings/normalized.yaml`. The normalizer merges two current-version claims for each component, preserves their scanner sources, and excludes a stale `legacy-pdf` version using the active-version inventory in policy. Two scanners reporting the same vulnerable component do not create two risks, while confidence and source disagreement remain inspectable.

> **Practice — Add deployed production context**
>
> Record configuration, reachability, exposure, known exploitation, affected assets, plausible harm, compensating controls, and owner.

Open `vulnerabilities/context.yaml` and `vulnerabilities/policy.yaml`. The reviewable policy expedites a deployed known-exploited flaw, any internet-reachable flaw, or a reachable flaw affecting a critical asset. It therefore does not equate internal with trusted or absence of observed exploitation with safety. The order parser is enabled, internet reachable, and known exploited. The PDF library remains deployed but its affected path is disabled. Neither severity label changed; the decision context did.

> **Practice — Select treatment with a deadline and proof**
>
> Remediate, mitigate, accept, or monitor through an owned record that says when and how the result will be verified.

`vulnerabilities/decisions.yaml` places the order parser first for remediation. Each record preserves decision time, deadline, uncertainty, owner, rationale, and the verification that later implementation must produce. The PDF finding remains second under monitoring rather than disappearing. Its exception records scope, rationale, compensating controls, evidence, expiry, and removal path.
Run:

```
make audit
make chapter-08-checkpoint
```

The checkpoint proves that raw claims reproduce the normalized inventory, every finding has context and an owned decision, the queue has unique contiguous priorities, urgent findings receive remediation within the policy window, overdue decisions fail, uncertainty remains explicit, and non-remediation treatments have live exceptions. It cannot prove exploitability or remediation against a live workload; production must obtain reachability, configuration, deployed-version, and outcome evidence from authoritative systems.

4. Test the decision under failure

Independent control failure — Severity outranks an exploitable order-path flaw

> **Practice — Reconcile scanner severity with production harm**
>
> Compare the critical unreachable library with the high, internet-reachable, known-exploited parser.

Severity: high; prioritization failure leaves a known-exploited order path exposed.
Plausible harm: unauthorized order-state manipulation and progression toward payment-related effects.
Potential blast radius: internet-originated requests reaching the vulnerable parser and downstream order processing.
Bounded by: rate limiting, expedited remediation, deployed-version verification, monitoring, and an owned deadline.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, and reconciliation.

Security questions

- **Asset and harm:** The correct order outcome makes the reachable parser more urgent than score ordering suggests.
- **Trust and authority:** Scanner labels provide evidence but have no authority to choose treatment without production context and ownership.
- **Detection after prevention fails:** Runtime inventory, request telemetry, exploitation intelligence, and verification evidence must expose changed reachability or attempted use.
- **Evidence of restored trust:** Not yet applicable; this chapter makes the remediation decision, while later implementation and deployment evidence must prove the vulnerable state was removed.

Diagnosis

Run `make chapter-08-challenge`. The command evaluates `checkpoints/chapter-08/cases/severity-only-decisions.yaml`, where the critical PDF finding is first and the order parser is demoted to monitoring. The policy rejects that decision for priority, treatment, and missing exception reasons independently.

Correction

Remediate the order parser under the urgent deadline and verify the fixed deployed version plus healthy order behavior. Keep the PDF exception only while its disabled configuration and import monitoring remain evidenced. Re-evaluate immediately if the module becomes reachable, exploitation emerges, compensating controls fail, or the exception expires.

5. Production reality

Best Practice: prioritize vulnerabilities through deployed exploitability and plausible harm while preserving uncertainty and time-bounded ownership.

Production Practice: correlate scanner results with artifact digests, SBOMs, runtime inventory, configuration, ingress paths, exploitation intelligence, and asset ownership. Measure decision age and overdue remediation rather than rewarding falling finding counts. A suppressive rule that hides recurring findings is not an exception lifecycle. **CISA (Cybersecurity and Infrastructure Security Agency)** maintains a **KEV (Known Exploited Vulnerabilities)** catalog as an authoritative input for exploitation in the wild; it informs urgency, it does not replace Northwind's reachability and harm decision. CISA KEV catalog.

Risk acceptance does not transfer accountability. It records why Northwind temporarily tolerates residual risk, who owns that decision, which controls compensate, what evidence remains required, and which event forces review.

6. What changed

Before	After
Scanner records competed by severity label.	**Normalized findings preserve identity, sources, and confidence.**
Presence in an image implied equal exposure.	**Deployed configuration, reachability, and exposure shape urgency.**
Critical automatically meant first.	**Exploitability and asset harm determine treatment order.**
Deferred work disappeared into backlog.	**Exceptions retain owners, controls, evidence, expiry, and removal.**
Closure meant changing ticket status.	**Each decision now specifies the evidence later implementation must produce before closure.**

Durable outputs

Artifact	Location	Keep it because
Prioritized remediation queue	vulnerabilities/policy.yaml and vulnerabilities/decisions.yaml	They preserve reviewable urgency rules plus context-backed treatment, uncertainty, ownership, deadlines, and planned verification.
Vulnerability exception record	vulnerabilities/exceptions.yaml	It makes deferred risk bounded, evidenced, expiring, and removable.

What You Learned

Severity is evidence about technical consequence, not a production priority. Northwind must correlate findings with deployed version, configuration, reachability, exposure, exploitation evidence, asset harm, compensating controls, and uncertainty. Unreachable does not mean harmless; it means the decision depends on evidence that must remain current.

Prove It

> **Independent Practice — Decide between two aging findings**
>
> Compare an internet-reachable medium flaw affecting authentication with a critical internal build tool flaw that requires administrative access.

Record deployed state, prerequisites, exploitation evidence, assets, harm, uncertainty, treatment, owner, deadline, exception requirements, verification, and the trigger that would reverse your priority.

Next

Vulnerability risk is now owned and prioritized. Chapter 9 governs the reusable secrets that can collapse otherwise bounded identity and supply-chain controls.

9. Govern Secrets Through Their Complete Lifecycle

Chapter 8 made vulnerability risk explicit. A reusable secret can still collapse the identity and supply-chain boundaries Northwind has built: whoever possesses the value may exercise its authority without inheriting the identity controls that should govern its use.

1. A secret found is not a secret recovered

Northwind's payment-provider credential is stored as a **CI (Continuous Integration)** variable and injected as plaintext. A scanner can identify its synthetic marker, but no inventory proves its owner, consumers, custody, active version, or revocation state.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-09-baseline

chapter 09 baseline: plaintext synthetic credential passed permissive
reference policy
```

Finding the string is only exposure evidence. Recovery requires Northwind to stop disclosure, remove old authority, replace it safely, inspect use, reconcile provider effects, and prove the exposed material no longer works.

2. The production model: govern authority through a lifecycle

> *Theory — Secrets, identities, custody, and rotation*
>
> This model enables Northwind to minimize unavoidable secrets and recover when their confidentiality fails.

An identity is an attributable subject with claims evaluated in context. A secret is material whose possession grants capability. A secret may authenticate a subject, decrypt data, sign artifacts, or authorize a provider call, but possession alone often weakens attribution because different holders can present the same value.

Lifecycle stage	Required decision	Evidence
Create	Is a reusable secret unavoidable, and how is it generated?	Purpose, owner, custodian, creation method
Store	Which system protects plaintext and encryption keys?	Approved custody and reference-only manifests
Distribute	Which consumers may receive which version?	Workload identity, reference, delivery decision
Use	Which subject exercised which purpose and authority?	Secret version, subject claim, result, time

Lifecycle stage	Required decision	Evidence
Rotate	How do old and new versions overlap without interruption?	Issue, overlap, consumer migration, health
Revoke	Which authority and derived material must stop working?	Provider rejection and access-history review
Retire	When is historical material no longer recoverable or usable?	Inventory state, backup and log constraints

Envelope encryption separates a data-encryption key from the key that protects it, improving rotation and custody boundaries. It does not eliminate the initial authority needed to obtain decryption capability. That bootstrap dependency is sometimes called secret zero. Federation and workload identity can reduce it, but every remaining root must be named rather than hidden.

Rotation is not replacement alone. During bounded overlap, old and new versions may both work so consumers can move safely. The overlap increases authority and must expire. Revocation closes old authority; retirement records that it is no longer an active operational version.

3. Enforce custody, reference-only use, and attribution

> **Practice — Inventory unavoidable secrets**
>
> Record purpose, owner, custodian, storage, consumers, versions, rotation method, and exceptions.

Open `secrets/inventory.yaml`. Northwind retains one modeled payment-provider credential because the external provider still requires it. `payment-v1` is retired; `payment-v2` is active under the secret broker. Workload federation should replace reusable credentials wherever the dependency supports it.

> **Practice — Keep plaintext out of governed manifests**
>
> Store only a broker reference and the expected version in workload configuration.

Open `secrets/policy.yaml` and `secrets/references.yaml`. The policy requires approved custody, ownership, reference-only use, attributable access, and no more than 15 minutes of rotation overlap. The reference manifest carries no usable value.

> **Practice — Preserve lifecycle and use evidence**
>
> Bind every modeled use to secret ID, version, workload subject, claim, purpose, result, and time.

The authorization mechanism emits `build/chapter-09-access-events.jsonl` when it evaluates the allowed `payment-v2` use by `order-worker`. The checkpoint cross-checks secret, version, subject, claim, purpose, and result rather than trusting a hand-written event. Run:

```
make audit
make chapter-09-checkpoint
```

The checkpoint validates governed artifacts, reference-only configuration, one active version, bounded overlap, an approved consumer, and attributable use. This deterministic model does not operate a real broker, encryption service, CI system, or payment provider; production must validate custody, delivery, access logs, masking, and provider rejection against those systems.

4. Test the design under failure

Connected consequence — Expose and replay a brokered payment credential

> **Practice — Treat exposure as loss of confidentiality, not proof of misuse**
>
> Exercise an inert marker representing `payment-v1` in a CI log and replay its modeled authority through the compromised-maintainer path.

Severity: critical; the exposed credential can authorize payment-provider effects.
Plausible harm: unauthorized charges, fraudulent reconciliation, concealed payment divergence, and customer harm.
Potential blast radius: provider operations authorized by the exposed credential until revocation.
Bounded by: masking, consumer policy, versioned rotation, provider revocation, attributable access, effect reconciliation, and replay tests.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Payment authority and attributable payment effects drive the response.
- **Trust and authority:** Possession of the old value must not grant continuing authority after revocation.
- **Detection after prevention fails:** Exposure location, version, subject claim, access history, provider operations, and masking status remain reviewable.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: old replay is denied, the new version works only for the approved workload, provider effects reconcile, and only the replacement remains accepted.

Diagnosis

Run `make chapter-09-attack`. The pre-rotation model demonstrates that the compromised maintainer can replay the exposed synthetic `payment-v1` authority and writes `build/chapter-09-exposure.yaml`. The modeled provider then records an unauthorized `payment-order-9001-authorized` effect, accepts both versions during the transition, reports degraded security health, and writes `build/chapter-09-provider-compromised.yaml` for recovery. No real credential or external provider is used.

Containment

Run `make chapter-09-contain`. It requires `build/chapter-09-exposure.yaml` from the attack, derives the affected version, records revocation in `build/chapter-09-revocations.json`, marks further CI-log disclosure as masked, confirms historical-access inspection, and records replacement of the derived provider session. Containment stops new modeled use; it does not prove that earlier provider effects are legitimate. Chapter 10 containment reads that revocation file.

Recovery

Run `make chapter-09-recover`. Recovery evaluates `payment-v1` against the pre-rotation inventory where it is still active, so denial depends on the containment revocation rather than its later retired label. It distinguishes unknown, retired, and revoked versions in its evidence. Recovery then proves `payment-v2` works for `order-worker`, verifies v1 is absent from provider acceptance after the bounded overlap, identifies the unauthorized provider effect, records its reversal, reconciles final effects, and requires healthy service. It writes `build/chapter-09-recovery.json`.

Historical repository content, logs, caches, backups, derived session tokens, and copied local files remain investigation scope even after provider revocation. Rotation without that scope assessment restores a credential but may leave other authority active.

5. Production reality

Best Practice: eliminate reusable secrets where federation is available; govern every unavoidable secret from creation through verified retirement.

Production Practice: separate secret custody from application configuration, use short-lived delivery, bind access to workload identity, test masking and broker outage, rotate under observed service health, and verify revocation at the authoritative provider. Maintain emergency procedures for broker or identity-provider failure without creating permanent plaintext fallbacks.

Secret scanning should cover current files, history, logs, artifacts, tickets, and generated output, but scanner results remain exposure indicators. High-entropy detection can miss structured credentials and produce false positives. Ownership and provider-side verification determine the response.

6. What changed

Before	After
A CI variable represented the payment credential.	**An inventory defines purpose, owner, custody, consumers, and versions.**
Workloads received plaintext configuration.	**Governed manifests contain broker references only.**
Rotation meant writing a new value.	**Issue, bounded overlap, migration, revocation, and retirement form one lifecycle.**
Exposure detection implied incident closure.	**Replay rejection and access-history scope prove authority was removed.**
Service success implied recovery.	**Replacement use and reconciled payment effects prove business and security recovery.**

Durable outputs

Artifact	Location	Keep it because
Secret inventory	secrets/inventory.yaml, secrets/policy.yaml, and secrets/references.yaml	They preserve ownership, custody, consumers, lifecycle, and reference-only use.
Rotation and exposure-response runbook	secrets/exposure-response-runbook.md	It preserves containment, rotation, revocation, historical inspection, derived-credential replacement, reconciliation, and recovery obligations.

What You Learned

A secret is authority-bearing material, not merely a sensitive string. Safe governance spans necessity, generation, custody, distribution, use, rotation, revocation, retirement, and evidence. Exposure response is complete only when old authority fails, replacement use remains healthy and attributable, and external effects reconcile.

Prove It

> **Independent Practice — Rotate signing material after suspected exposure**
>
> Design the response without treating a newly generated key as proof that old release authority disappeared.

Specify custodian, affected signatures and derived trust, overlap policy, revocation distribution, transparency evidence, rebuild scope, old-key rejection, new-key use, historical inspection, and the observation that proves the trust migration closed.

Next

Authority-bearing secrets are now governed. Chapter 10 protects customer and operational data according to purpose, sensitivity, retention, deletion, and permitted use.

10. Protect Data According to Its Use and Sensitivity

Chapter 9 governed authority-bearing secrets. Northwind still holds customer and operational data whose misuse may be fully authenticated and encrypted. Protection must therefore govern why data exists, who may use which fields, where copies may live, and how every copy reaches deletion.

1. Encryption does not prevent authorized overexposure

The notification service needs an order identifier, customer email, and total to send a confirmation. A malicious dependency requests `payment_reference` as well, then attempts to place the expanded record in telemetry and a non-production fixture. Encryption at rest would protect neither authorized retrieval nor excessive copying.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-10-baseline
```

```
chapter 10 baseline: permissive purpose policy admitted payment data to telemetry
```

2. The production model: sensitivity follows use and harm

> *Theory — Classification, purpose, minimization, and lineage*
>
> This model enables Northwind to derive controls from the harm a field can cause in a particular use.

Classification labels the protection need of a data element. Purpose explains the permitted business reason for processing it. Minimization asks whether that purpose can be achieved with fewer fields, less precision, fewer copies, or shorter retention.

Decision	Question	Control consequence
Classification	What harm follows disclosure or alteration?	Access, storage, telemetry, and non-production limits
Purpose	Why may this subject process the data?	Field-level authorization and decision evidence
Store	Where may this use create state?	Encryption context, residency, retention, and deletion
Transformation	Can masking or tokenization preserve utility with less authority?	Reduced values and separated mapping authority
Retention	How long does the purpose remain valid?	Store-specific expiry and purge

Decision	Question	Control consequence
Lineage	Which derived copies and backups inherit obligations?	Propagated classification, deletion, and restore controls

Encryption protects data under particular key and access assumptions. Tokenization replaces a sensitive value with a reference and separates the mapping authority. Masking reduces displayed precision. None decides whether collection was necessary or whether an authenticated support user may see the field.

Deletion is a distributed state transition. Primary data, telemetry, caches, non-production copies, exports, and backups have different mechanisms. An immutable backup may expire by generation rather than support immediate record rewriting; every restore must reapply current tombstones before the data becomes usable.

3. Bind permitted fields to subject, purpose, and store

> **Practice — Classify data by plausible harm**
>
> Assign an owner and sensitivity to each governed field.

Open `data-security/classification.yaml`. `payment_reference` is restricted because disclosure creates payment linkage and fraud risk. `customer_email` and `order_total` are confidential; `order_id` is internal.

> **Practice — Declare minimum permitted uses**
>
> Bind each purpose to a subject, minimum field set, and allowed stores.

Open `data-security/uses.yaml` and `data-security/access-policy.yaml`. Notification may use three fields only in runtime memory. Support inquiry receives a narrower support-session view—the control that Chapter 2's `overprivileged-support-data-access` path requires for `governed-order-data`. Payment reconciliation belongs to `order-worker`, not notification or support. The access policy gives every known destination an explicit maximum class; an unknown or misspelled store is rejected rather than silently operating without a ceiling.
The field sets are reviewed claims of minimum need. The evaluator can prove that a request does not exceed a declared use, but it cannot prove that the declared use itself contains the fewest fields capable of achieving the business purpose.

> **Practice — Govern lifecycle by store**
>
> Set retention and deletion behavior for primary, telemetry, non-production, and backup copies.

`data-security/retention.yaml` makes each deletion mechanism explicit for primary, runtime memory, support sessions, telemetry, non-production, and backups. `data-security/lineage.yaml` maps derived copies from their source to destination and inherits the destination deletion contract. The checkpoint rejects any used or derived store without retention, deletion, and class policy. Run:

```
make audit
make chapter-10-checkpoint
```

The checkpoint proves the minimum notification use remains available and lifecycle contracts are complete. This local model cannot prove deletion from a real database or backup system; production needs authoritative access decisions, store inventories, purge reports, restore tests, and exception evidence.

4. Test the design under failure

Connected consequence — Copy payment-related data into logs and non-production

> **Practice — Deny unnecessary fields and unsafe destinations**
>
> Evaluate the malicious dependency's request for `payment_reference` and telemetry storage.

Severity: high; the request expands restricted payment-linked data beyond its declared purpose.
Plausible harm: customer disclosure, payment linkage, fraud enablement, and uncontrolled secondary copies.
Potential blast radius: telemetry consumers, retained log stores, and non-production users receiving the copied record.
Bounded by: field-purpose authorization, store class limits, minimization, quarantine, purge, retention, and secret rotation where authority was exposed.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Customer order data, payment authority, and correct order outcomes drive the boundary.
- **Trust and authority:** Notification authority grants no payment-reconciliation or telemetry-copy authority.
- **Detection after prevention fails:** The decision retains subject, purpose, fields, classes, store, result, and named denials; lifecycle evidence identifies copied stores.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: governed copies are purged, exposed authority is rotated where relevant, sanitized replacements contain no production values, and backup expiry constraints remain explicit.

Diagnosis

Run `make chapter-10-attack`. The decision independently rejects the undeclared payment field, every confidential or restricted field that exceeds telemetry's class ceiling, and the undeclared telemetry store. It writes the decision plus `build/chapter-10-exposed-copies.json`, which materializes the synthetic telemetry and non-production copies used by containment.

Containment

Run `make chapter-10-contain`. It requires the denial and exposed-copy state, derives quarantine and purge scope from those copy IDs, and records an empty remaining-copy set. It also reads `build/chapter-09-revocations.json` and requires the exposed payment version to be

revoked instead of accepting a descriptive cross-chapter label. If that file is absent, run `make chapter-09-attack chapter-09-contain` first.

Recovery

Run `make chapter-10-recover`. Recovery reconciles every exposed copy ID against the purge transition, then validates each sanitized-fixture field through `classification.yaml` and the non-production class ceiling. It independently compares fixture values with the exposed synthetic values, permitting any classified public or internal fields without relying on a self-declared class list. The recovery record distinguishes immediate governed-store purge from backup expiry and restore-time tombstone reapplication.

5. Production reality

Best Practice: authorize the smallest field set for one declared purpose and propagate its lifecycle obligations to every derived store.

Production Practice: enforce access near authoritative data, preserve application context in decision logs, tokenize payment-linked values, prevent restricted classes from entering telemetry, generate synthetic non-production fixtures, and reconcile deletion across indexes, caches, exports, analytics, and backups. Test restored backups against current tombstones before releasing them.

Classification must change when purpose, harm, regulation, topology, or transformations change. Labels copied once into a catalog and never reconciled with actual stores become misleading evidence.

6. What changed

Before	After
Encryption represented complete protection.	**Purpose, field, subject, and store govern authorized use.**
Services received broad order objects.	**Minimum declared fields bound each use.**
Telemetry inherited application data accidentally.	**Store class limits reject sensitive copies.**
Non-production reused production-shaped values.	**Sanitized synthetic fixtures preserve utility without production data.**
Deletion referred only to the primary database.	**Every store has retention, purge, backup, and restore obligations.**

Durable outputs

Artifact	Location	Keep it because
Data classification and use register	data-security/classification.yaml and data-security/uses.yaml	They preserve harm, ownership, purpose, subject, minimum fields, and stores.
Retention and deletion contract	data-security/access-policy.yaml, data-security/retention.yaml, and data-security/lineage.yaml	They bind store ceilings, derived copies, deletion behavior, and backup limitations.

What You Learned

Data protection is governance of permitted use, not encryption alone. Classification, purpose, minimization, transformations, store boundaries, retention, deletion, and lineage must agree. Recovery removes unsafe copies, replaces them with validated sanitized data, and documents where immutable backup expiry delays physical deletion.

Prove It

> **Independent Practice — Design a support investigation view**
>
> Let support diagnose a failed order without granting general customer-history or payment-data access.

Specify subject, purpose, fields, masking, session store, retention, evidence, approval, exception behavior, deletion, and the observation that would reveal over-collection.

Next

Identity, supply chain, vulnerabilities, secrets, and data now have domain policies. Chapter 11 places those policies at explicit enforcement points and governs exceptions without hiding them.

11. Enforce Security Policy Without Hiding Exceptions

Chapters 4 through 10 created policies for identity, privilege, source, release, vulnerabilities, secrets, and data. A policy document changes nothing unless Northwind places it at a boundary that can decide, records what happened, and prevents exceptions from becoming invisible permanent authority.

1. Release pressure becomes policy

A release manager proposes one broad waiver that disables dependency and production-admission checks, covers every change, has no compensating detection, and never expires. No attacker is required; organizational pressure alone can reopen the compromised-maintainer path.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-11-baseline

chapter 11 baseline: permissive exception policy admitted a placebo
release waiver
```

2. The production model: intent, placement, decision, exception

> *Theory — Enforcement points and failure behavior*
>
> This model enables Northwind to turn policy intent into consistent decisions without hiding operational trade-offs.

Policy intent states the outcome. A rule makes a testable condition. An enforcement point has the context and authority to allow, deny, transform, or observe a transition. Decision evidence binds input identity, rule, policy version, result, reason, and exception.

Choice	Appropriate use	Principal risk
Admission enforcement	Unsafe state must not cross a boundary	Verifier outage can block safe work
Runtime enforcement	Context exists only during use	Latency or availability couples to policy engine
Detection-only	Prevention is impossible or too risky	Harm may occur before response
Fail-closed	Missing decision is more dangerous than interruption	Availability loss
Fail-open	Continuity is more important for this bounded transition	Silent unsafe admission

Central policy improves consistency and review but can lose local context. Local policy understands its system but can drift or be bypassed. Northwind therefore governs a versioned bundle while declaring where each rule is enforced and which context that point supplies.

An exception is a bounded policy decision, not the absence of policy. It needs an owner, rationale, narrow scope, affected enforcement points, compensating controls, evidence, effective time, expiry, and removal path.

3. Place rules and govern deviations

> **Practice — Map every rule to an enforcement point**
>
> Declare where source, deployment, data, and detection decisions occur.

Open `policy/bundle/rules.yaml` and `policy/enforcement-points.yaml`. Dependency policy runs at source merge, release policy at production deployment, and field policy at data query. The `runtime-observation` point is detection-only for `data-field-access`; Chapter 12 still prevents shell execution and undeclared egress. Every point requires a decision log and names its failure mode.

> **Practice — Make exceptions visible and expiring**
>
> Preserve scope, ownership, compensation, evidence, expiry, and removal.

Open `policy/exception-policy.yaml` and `policy/exceptions.yaml`. The governance policy caps exceptions at four hours, forbids wildcard and release-wide scopes, rejects placeholder compensation, requires independent compensation points and multiple evidence items, and marks production release admission non-exceptable. A registry-mirror outage permits one locked dependency at source merge for two hours. Hash verification and runtime resolution alerts compensate at a point independent of source merge.

The evaluator uses the explicit `evaluation_time` in exception policy so exercises remain deterministic. Production evaluation must use a trustworthy current clock and preserve that instant in its decision evidence.

Run:

```
make audit
make chapter-11-checkpoint
```

The checkpoint consumes each rule's `unsafe_result`: a deny rule must have at least one fail-closed placement, not merely a detection point. It also verifies supported failure behavior, required logging, and exceptions against the reviewable governance policy. The lab does not deploy a repository rule, admission controller, or distributed policy service; production must verify real enforcement and decision delivery under dependency failure.

4. Test the design under failure

Independent control failure — Release pressure creates a permanent bypass

> **Practice — Reject organizationally created trust expansion**
>
> Evaluate a broad, non-expiring exception that disables dependency and release admission.

Severity: critical; the bypass removes independent supply-chain enforcement.
Plausible harm: untrusted dependency or artifact admission, production compromise, and concealed release authority.
Potential blast radius: every source change and production deployment covered by the wildcard.
Bounded by: rule-specific scope, named enforcement points, compensation, evidence, expiry, logging, and automatic removal.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Release authority and correct production outcomes drive rejection.
- **Trust and authority:** Delivery urgency grants no authority to erase dependency and admission policy.
- **Detection after prevention fails:** Generated decision logs identify requester context, point, rule, policy version, denial reasons, and exception reference.
- **Evidence of restored trust:** Not yet applicable. Chapter-local correction: the broad bypass is absent, any narrow exception expires, and ordinary enforcement produces consistent decisions again.

Diagnosis

Run `make chapter-11-attack`. The evaluator independently rejects the wildcard scope, unknown all-rules target, missing compensation, missing evidence, absent expiry, and absent removal path. It writes the attempted bypass into generated exception state, emits `build/chapter-11-attack-decision.json`, and appends the requester-bound decision to `build/chapter-11-decisions.jsonl`.

Containment

Run `make chapter-11-contain`. It replaces the bypass in generated state with the reviewed mirror exception: one rule, one point, one dependency, independent compensating detection, evidence, and two-hour expiry. The generated decision preserves requester context, policy version, and exception reference.

Recovery

Run `make chapter-11-recover`. Recovery advances beyond the exception deadline, proves it is rejected as expired, transitions generated exception state to an empty list, verifies both the broad bypass and narrow exception IDs are absent, and evaluates the bundle with no exceptions to prove blocking enforcement resumes.

5. Production reality

Best Practice: place policy at the boundary with sufficient context and authority, then make every deviation narrower and shorter than the rule it relaxes.

Production Practice: version bundles, test decisions before rollout, retain allow and deny evidence, define verifier-outage behavior per point, measure exception age, alert before expiry, and reconcile configured points with actual infrastructure. Emergency policy must remain usable during the failure it addresses without becoming a general fail-open switch. Kubernetes admission controllers illustrate one production enforcement point: they can deny objects before persistence, and their failure mode is a cluster-level availability decision. Admission controllers.

6. What changed

Before	After
Policies existed as disconnected files.	**Every rule maps to a named enforcement point.**
Failure behavior was accidental.	**Fail-closed and detection-only choices are explicit.**
Decisions lacked a stable interpretation.	**Policy version, point, result, reason, and exception are retained.**
Waivers disabled broad control families.	**Exceptions are narrow, owned, compensated, evidenced, and expiring.**
Removing a ticket implied recovery.	**Normal enforcement is re-evaluated without bypass state.**

Durable outputs

Artifact	Location	Keep it because
Enforcement-point map	policy/bundle/rules.yaml and policy/enforcement-points.yaml	They preserve rule ownership, placement, failure behavior, and evidence obligations.
Exception register	policy/exceptions.yaml	It keeps deviations scoped, owned, compensated, evidenced, expiring, and removable.

What You Learned

Policy becomes security only at a real decision boundary. Central intent and local context must meet in versioned, reviewable decisions. Exceptions are temporary governed decisions whose scope, compensation, evidence, expiry, and removal are as important as their rationale.

Prove It

> **Independent Practice — Design an identity-provider outage exception**
>
> Preserve one legitimate containment action without converting an authentication outage into general fail-open production access.

Specify point, rule, subject, action, resource, duration, evidence, compensation, approver, logging, expiry, removal, and the check that proves ordinary identity enforcement resumed.

Next

Policy now governs expected transitions and visible exceptions. Chapter 12 constrains workloads that reach runtime and detects behavior outside their process, filesystem, identity, privilege, and network contracts.

12. Constrain Workloads and Detect Runtime Abuse

Chapter 11 placed policy at explicit boundaries. A policy-admitted artifact can still behave maliciously after startup through an exploitable path, compromised dependency, stolen identity, or undeclared runtime input. Northwind must bound what each workload can do and observe meaningful contract violations independently.

1. Healthy service, malicious behavior

The order worker remains available and processes orders while a malicious dependency searches for credentials, launches a shell, and attempts outbound command-and-control traffic. Health checks answer whether the process responds; they do not answer whether its behavior remains authorized.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-12-baseline

chapter 12 baseline: detection-only runtime policy allowed shell execution
to proceed
```

Detection is necessary but cannot substitute for prevention where a high-confidence action has no legitimate runtime use.

2. The production model: constrain and observe complementary behavior

> *Theory — Runtime contracts, sandbox boundaries, and behavioral evidence*
>
> This model enables Northwind to preserve required workload behavior while bounding compromise.

A workload contract defines identity, artifact, process, privilege, filesystem, and network behavior expected for one workload. A sandbox enforces some boundaries; a runtime sensor observes behavior. Neither is complete alone.

Dimension	Contract question	Failure consequence
Identity	Which workload subject may act?	Stolen or confused authority
Privilege	Which capabilities and escalation are required?	Host or neighboring workload impact
Process	Which executables and children are expected?	Shells, tooling, or injected processes
Filesystem	Which paths may be read or written?	Credential discovery or persistence
Egress	Which destinations serve declared dependencies?	Exfiltration or command and control
Artifact	Which admitted digest owns this behavior?	Events cannot be tied to a release

Prevention suits narrow, high-confidence prohibitions such as privilege escalation, undeclared egress, and shell execution in this worker. Detection suits behavior that may have legitimate variants or where blocking could destroy evidence. Defense in depth means one control's failure does not silently grant the full workload authority.

Behavioral baselines are hypotheses, not universal normality. Too broad a baseline normalizes attacker behavior; too narrow a baseline produces false positives and operational bypass. Evasion remains possible when sensors lack visibility or attackers mimic permitted actions.

3. Define and verify the runtime contract

> **Practice — Bound workload authority**
>
> Declare identity, digest, processes, writable paths, egress, capabilities, escalation, and root-filesystem behavior.

Open `runtime/contracts/order-worker.yaml`. The worker has no Linux capabilities, cannot escalate, uses a read-only root filesystem, writes only to declared ephemeral paths, and reaches only PostgreSQL, payment, and email dependencies. The required dependency list is part of the contract, so removing any declared operational dependency fails verification without hardcoded evaluator knowledge.

> **Practice — Separate prevention from detection**
>
> Assign an explicit response mode to each meaningful violation.

Open `runtime/policies/behavior.yaml`. Shell execution, privilege escalation, root writes, and undeclared egress are prevented. Credential discovery is detected so evidence remains available for investigation. Every event requires subject and deployment context.

Run:

```
make audit
make chapter-12-checkpoint
```

The checkpoint sends a declared process, an allowed ephemeral write, and all required egress destinations through the same behavior evaluator used by the attack. Each must produce `allowed`, while static contract checks retain identity and privilege constraints. This model does not exercise a kernel sensor, container runtime, or cluster network; production must verify actual enforcement, sensor completeness, performance, evasion resistance, and false-positive behavior.

4. Test the design under failure

Cumulative attack — Discover credentials, execute a shell, and call outbound

> **Practice — Exercise the malicious dependency's runtime behavior**
>
> Generate inert observations without executing a shell, reading credentials, or making network traffic.

Severity: critical; the behavior seeks credentials, execution capability, and external control.
Plausible harm: secret exposure, payment authority misuse, data exfiltration, persistence, and production manipulation.
Potential blast radius: order-worker authority plus any reachable dependency not bounded by runtime policy.
Bounded by: least privilege, read-only filesystem, process policy, egress allowlist, workload identity, artifact attribution, isolation, and revocation.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Order outcomes, payment authority, secrets, and customer data drive confinement.
- **Trust and authority:** Admission authorizes one artifact to perform its workload contract, not arbitrary execution or egress.
- **Detection after prevention fails:** Runtime events bind subject, claim, action, resource, outcome, policy, deployment, digest, correlation, and sensitivity.
- **Evidence of restored trust:** Not yet applicable. Chapter-local recovery: the workload is isolated, identity revoked in generated state, replacement digest admitted, legitimate processing verified, and related behavior absent under active monitoring.

Diagnosis

Run `make chapter-12-attack`. Raw observations name behavior type and resource, not a preselected violation label. The evaluator compares `/bin/sh` with allowed processes, the secret path with filesystem expectations, and `c2.invalid` with allowed egress. Credential discovery is detected; shell execution and undeclared egress are blocked. The command emits `runtime/events.jsonl` from the mechanism so Chapter 13 can consume the runtime evidence rather than recreate it.

Containment

Run `make chapter-12-contain`. The `order-worker` subject already exists in the Chapter 4 identity inventory. Containment requires all three runtime events, reads that active subject, writes `build/chapter-12-contained-subjects.yaml` with the subject revoked, records the exact compromised claim, isolates the worker, preserves evidence, and requires artifact replacement. It does not mutate the operational identity register.

Recovery

Run `make chapter-12-recover`. Recovery binds the runtime contract to Chapter 7's admitted deployment digest, then evaluates a replacement process, allowed write, and every required egress destination. The generated record retains those allowed observations, derives a zero violation count, and states that coverage is limited to the declared observations under active monitoring.

5. Production reality

Best Practice: build workload-specific contracts from required behavior, prevent high-confidence prohibited actions, and retain independently attributable runtime evidence.

Production Practice: remove capabilities, prohibit escalation, use read-only roots and bounded writable volumes, restrict egress by identity and destination, protect metadata endpoints, correlate events with deployment digests, and test enforcement during rollout. Stage new rules in observation mode, measure false positives, then promote only well-understood prohibitions to blocking. Kubernetes documents `securityContext` controls for capabilities, privilege escalation, and a read-only root filesystem. Configure a security context. Cloud instance-metadata endpoints are a common credential source; AWS, for example, documents that **IMDSv2 (Instance Metadata Service version 2)** requires a session-oriented request rather than an unauthenticated GET. Instance metadata and user data.

6. What changed

Before	After
Container admission implied runtime trust.	**One admitted digest receives one bounded runtime contract.**
Generic hardening applied equally to every service.	**Identity, process, filesystem, privilege, and egress are workload-specific.**
Detection covered whatever the sensor happened to emit.	**Meaningful behaviors have explicit prevent or detect outcomes.**
Runtime alerts lacked release context.	**Events bind workload claims to deployment and artifact digest.**
Restarting the worker implied recovery.	**Isolation, revocation, replacement, behavior, and monitoring prove recovery.**

Durable outputs

Artifact	Location	Keep it because
Runtime security contract	runtime/contracts/order-worker.yaml and runtime/policies/behavior.yaml	They preserve required authority and explicit prevent/detect behavior.
Containment mini-runbook	runtime/isolation-runbook.md	It preserves isolation, exact identity revocation, evidence, replacement, behavioral verification, and coverage obligations across fresh clones.

What You Learned

Runtime trust is conditional behavior, not successful deployment. Workload identity, privilege, process, filesystem, egress, and artifact context form one contract. Prevention bounds high-confidence abuse; detection preserves evidence for behavior that cannot safely be blocked. Recovery replaces compromised authority and verifies legitimate operation under continued observation.

Prove It

> **Independent Practice — Contract notification-service runtime behavior**
>
> Permit email delivery without granting shell execution, payment access, broad filesystem writes, or unrestricted egress.

Specify identity, digest, processes, destinations, writable paths, capabilities, prevented actions, detected actions, event context, containment, and recovery evidence.

Next

Runtime events now exist with identity and deployment context. Chapter 13 normalizes cross-system evidence, builds detection hypotheses, exposes telemetry gaps, and connects actionable detections to response.

13. Build Security Evidence and Actionable Detections

Chapter 12 produced attributable runtime events. Northwind also has identity denials, supply-chain decisions, privilege attempts, secret evidence, and data decisions. Separate alerts do not reveal whether those observations form one intrusion or unrelated operational noise.

1. Four signals, no shared meaning

A credential misuse denial, unusual dependency change, denied elevation, and blocked runtime egress exist in four systems. Each looks locally contained. Together they describe the compromised-maintainer path progressing from identity through source and privilege to runtime.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-12-attack chapter-13-baseline

chapter 13 baseline: four valid signals had no detection hypothesis and
produced no alert
```

`chapter-12-attack` writes the undeclared-egress event in `runtime/events.jsonl` that the Chapter 13 evaluator joins with the modeled identity, supply-chain, and privilege observations.

2. The production model: detect hypotheses, not tool activity

> *Theory — Evidence, correlation, coverage, and telemetry integrity*
>
> This model enables Northwind to turn observations into an owned response decision.

A signal is an observation that may matter. Evidence is a signal interpreted with producer, subject, time, integrity, provenance, and limitations. A detection hypothesis states which attack behavior should produce which evidence, within what window, with which context, and what response follows.

Element	Required question	Failure mode
Event contract	Can sources preserve common identity and deployment context?	Parallel security telemetry loses operational traces
Hypothesis	Which plausible path should the evidence reveal?	Rules measure generic suspiciousness
Correlation	Which shared fields and time window join events?	Unrelated activity becomes one incident
Threshold	Which distinct observations are sufficient?	Repeated noise satisfies a count
Integrity	Can evidence loss or alteration be detected?	Missing telemetry is interpreted as safety
Routing	Who acts, and what action is expected?	Alerts accumulate without reducing harm
Coverage	Which path steps remain unobserved?	Dashboards imply universal visibility

False positives consume response capacity and encourage suppression. False negatives hide harmful behavior. Threshold tuning must preserve the threat hypothesis: requiring distinct actions within a bounded window is safer here than counting repeated authentication denials.

Northwind extends the inherited DevOps telemetry contract rather than building a security-only stack. Deployment identity, traces, service outcomes, and security decisions must remain correlatable.

3. Normalize evidence and encode response hypotheses

> **Practice — Define one security event contract**
>
> Require core event fields, then require action-specific identity, session, deployment, and artifact context.

Open `detection/event-contract.yaml`. It defines required core fields, accepted integrity state, retention, and missing-telemetry behavior. The rule adds the context required for each action. Missing or unacceptable context creates a telemetry-gap finding; incomplete events are not silently treated as evidence of absence.

> **Practice — Map the priority attack path to evidence**
>
> Declare required distinct actions, permitted join fields, time window, owner, and response.

Open `detection/hypotheses.yaml` and `detection/rules/correlate-progression.yaml`. The cumulative hypothesis requires four distinct actions within 30 minutes. The evaluator computes the join: identity, dependency, and privilege evidence share the maintainer session; the dependency evidence then bridges to Chapter 12's emitted runtime event through the admitted artifact digest. No preassigned incident identifier manufactures the relationship. The result routes to `security-response` with the action `investigate-and-isolate-order-worker`.

Run:

```
make audit
make chapter-13-checkpoint
```

The checkpoint proves the controlled event set—including the event emitted by Chapter 12—normalizes, correlates, fires, and carries ownership plus response. It also reads the retention and accepted-integrity contract. It does not benchmark a production telemetry pipeline or prove resistance to attacker evasion; real deployments must validate delivery latency, cryptographic or platform-backed integrity, retention enforcement, clock behavior, source coverage, and routing.

4. Test the design under failure

Cumulative attack — Correlate identity, dependency, privilege, and runtime evidence

> **Practice — Detect progression rather than count alerts**
>
> Replay inert normalized observations for the cumulative compromised-maintainer path.

Severity: critical; correlated evidence shows progression toward production and payment-related authority.
Plausible harm: malicious release, privilege escalation, secret discovery, exfiltration, and fraudulent business effects.
Potential blast radius: identities, delivery systems, and runtime resources linked by the intrusion correlation.
Bounded by: normalized context, distinct-action threshold, bounded window, telemetry-gap alarms, owned routing, and workload isolation.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Release authority, payment authority, secrets, data, and order outcomes drive the hypothesis.
- **Trust and authority:** No individual denial proves safety; correlated attempts reveal how authority is being pursued.
- **Detection after prevention fails:** Independent identity, supply-chain, privilege, and runtime producers contribute evidence with integrity and deployment context.
- **Evidence of restored trust:** Not yet applicable. Chapter-local correction: the controlled path still alerts after context correction and noise tuning, with complete response context and explicit coverage limits.

Diagnosis

Run `make chapter-13-attack`. The four observations correlate into `build/chapter-13-alert.json`, carrying distinct actions, window, owner, response, and evidence count.

Containment

Run `make chapter-13-contain`. It persists a damaged event set with artifact context removed from the dependency event. The normalizer raises a telemetry-gap alarm, excludes the incomplete event, and the cumulative path correctly produces no alert. This demonstrates the detection loss honestly: the independent gap alarm preserves an owned signal, but it does not replace the suppressed attack-path detection.

Recovery

Run `make chapter-13-recover`. Recovery reads the persisted damaged event set, restores its artifact digest from `supply-chain/deployment-evidence.yaml`, writes the corrected evidence, and requires that corrected set to produce one actionable alert with all four evidence items. The record states that proof covers only modeled sources and actions; it does not claim universal attacker visibility.

5. Production reality

Best Practice: begin with a plausible threat hypothesis, require sufficient context for action, and alarm independently when required evidence disappears.

Production Practice: normalize into the existing telemetry path, preserve raw provenance, protect clocks and retention, monitor source delivery, test routing, measure time to actionable context, and tune with controlled simulations. Never optimize alert count by suppressing the only evidence covering a priority path. OpenTelemetry remains the inherited signal model; security events must remain correlatable with service and deployment context. OpenTelemetry signals, context propagation.

6. What changed

Before	After
Security events remained tool-specific.	**One contract preserves identity, deployment, policy, and integrity context.**
Alert counts represented detection maturity.	**Threat hypotheses define distinct required evidence.**
Missing events looked like quiet systems.	**Telemetry gaps alarm independently.**
Thresholds counted repeated noise.	**Distinct actions and bounded time establish progression.**
Alerts lacked operational consequence.	**Every hypothesis names an owner and response action.**

Durable outputs

Artifact	Location	Keep it because
Detection catalog	detection/event-contract.yaml, detection/hypotheses.yaml, and detection/rules/	They preserve event semantics, hypotheses, thresholds, ownership, and response.
Attack-path coverage report	Generated build/chapter-13-recovery.json plus the hypothesis mapping	It records controlled coverage, complete context, and explicit limitations. Regenerate it from the Chapter 13 recover command; it is not a substitute for the committed detection catalog.

What You Learned

Detection engineering connects plausible attack behavior to trustworthy evidence and a specific response. Correlation requires shared identity, deployment, time, integrity, and policy context. Missing telemetry is a finding, repeated noise is not progression, and controlled simulation proves only the declared coverage.

Prove It

> **Independent Practice — Detect payment-credential misuse**
>
> Correlate secret exposure, unexpected provider use, identity context, and payment-effect divergence without alerting on every ordinary payment request.

Specify required events, correlation, window, integrity, missing-source alarm, owner, response, false-positive boundary, retention, and coverage limitation.

Next

Northwind can now raise an evidence-rich signal. Chapter 14 investigates and contains the production compromise without destroying evidence or widening business harm.

14. Investigate and Contain a Production Compromise

Chapter 13 connected four observations into an actionable alert. An alert is not an incident history, proof of scope, or permission to destroy the suspected systems. Northwind must stop ongoing harm while preserving enough trustworthy evidence to learn what happened.

1. An alert exists, but the incident remains open

The compromised maintainer session is still active, the release path may still accept attacker-controlled work, the admitted artifact remains associated with `order-worker`, and orders are in flight. Immediate deletion could erase volatile evidence and business state. Investigation without containment would leave authority open.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-12-attack chapter-13-attack chapter-14-open chapter-14-baseline

chapter 14 baseline: correlated alert exists, but attacker authority and incident scope remain open
```

The baseline needs `runtime/events.jsonl` from Chapter 12, `build/chapter-13-alert.json` from Chapter 13, incident `INC-2026-0815-01` in `investigating`, and `compromised-session` still `active`. After Chapter 15 the committed case is `closed` and that session is revoked; `chapter-14-open` restores the investigation start state.

2. The production model: preserve, reason, then constrain

> *Theory — Incident hypotheses, evidence custody, scope, and staged containment*
>
> This model lets Northwind act quickly without turning assumptions into facts or erasing the evidence required for recovery.

An incident hypothesis is a testable explanation of observed harm. Facts are directly supported observations. Inferences connect facts but remain revisable. Unknowns define the investigation's current limits. Keeping those categories separate prevents urgency from creating false certainty.

Element	Required question	Failure mode
Hypothesis	What explanation is being tested?	Investigation becomes an unbounded search
Timeline	In what attributable order did events occur?	Correlation is mistaken for causation
Scope	Which identities, artifacts, workloads, data, and business effects may be affected?	Containment is too narrow or needlessly broad

Element	Required question	Failure mode
Provenance	Where did each evidence item originate?	Claims cannot be reproduced
Integrity	Has evidence changed since collection?	Altered material supports false conclusions
Volatility	Which evidence disappears first?	Destructive action erases the best lead
Containment	Which authority must close now?	Investigation leaves active attacker capability
Continuity	Which business state must be preserved?	Security action creates additional customer harm

Containment is not eradication. It closes identified access and execution paths while keeping recovery choices open. Attacker access is current authority; persistence is a mechanism that may restore authority later. Chapter 14 closes known access. Chapter 15 searches for and removes persistence before declaring trust restored. Current **NIST (National Institute of Standards and Technology)** incident guidance treats detection, response, and recovery as part of cybersecurity risk management rather than a closed four-phase procedure. Northwind still separates containment from eradication because they are different trust decisions. NIST SP 800-61r3.

Communication follows evidence and impact. Internal responders and service owners need early operational facts. Legal, privacy, payment, or customer-notification escalation follows the organization's defined thresholds; responders should preserve relevant evidence without inventing legal conclusions.

3. Build the investigation record and staged containment plan

> **Practice — Separate fact, inference, and unknown**
>
> Record one testable incident hypothesis without presenting suspected persistence or business harm as established fact.

Open `response/case/incident.yaml`. Case `INC-2026-0815-01` initially states the correlated observations as facts, builder control and payment divergence as inference or unknown, and assigns an owner. The checkpoint then collects a scoped order/payment reconciliation record. That evidence resolves the sampled payment-divergence unknown into a fact while leaving broader persistence unknown.

> **Practice — Preserve attributable evidence before mutation**
>
> Hash each collected artifact, record its provenance and custodian, and order the timeline by event time.

Run `make chapter-14-checkpoint`. It copies the Chapter 13 alert, Chapter 12 runtime events, identity register, deployment evidence, and payment reconciliation into `response/evidence/`, then creates `response/evidence-manifest.yaml` over those preserved snapshots. It also writes `response/timeline.jsonl`. Later response steps recalculate the snapshot hashes before trusting the evidence or mutating operational state.

> **Practice — Stage containment across every identified authority**
>
> Preserve evidence first; then revoke the session, freeze release, retain queue and database state, and isolate the workload.

Open `response/containment-plan.yaml`. Its explicit order prevents mutation before evidence preservation and retains in-flight business state before workload isolation. `bounded-intake` preserves defensible service without allowing the compromised worker or release path to resume normal authority.

4. Test the design under failure

Cumulative attack — Retained authority and unverified evidence

> **Practice — Reject an investigation claim built on changed evidence**
>
> Compare the collected digest with the evidence manifest before using the artifact to justify scope or recovery.

Severity: critical; attacker authority remains active while evidence quality determines every later decision.
Plausible harm: continued malicious releases, lost forensic scope, fraudulent effects, or destructive over-containment.
Potential blast radius: maintainer identity, build and release authority, production workload, in-flight orders, payment effects, and incident evidence.
Bounded by: preserved evidence, explicit uncertainty, staged authority closure, workload isolation, and retained business state.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Release authority, payment authority, order outcomes, and trustworthy incident evidence are at risk.
- **Trust and authority:** The session, release path, artifact, and workload are treated as affected until evidence narrows scope.
- **Detection after prevention fails:** The Chapter 13 alert initiates investigation; timeline and custody records support response decisions.
- **Evidence of restored trust:** Not yet applicable. This chapter proves containment, not eradication or restored production trust.

Diagnosis

Run `make chapter-14-attack`. The controlled failure copies one preserved item, mutates that working copy, and verifies it against the digest recorded at collection. The resulting `digest-mismatch` rejects the investigation claim while leaving the preserved snapshot intact. A missing item would instead produce a distinct `missing` finding.

Containment

Run `make chapter-14-contain`. The evaluator verifies custody before revoking `compromised-session` in the identity register, freezing the production deployment enforcement point, and isolating the `order-worker` runtime contract. It re-reads those controls, calls Chapter 4 authorization to prove the revoked session is denied, and proves a legitimate maintainer remains allowed for bounded source intake.

Recovery

Run `make chapter-14-recover`. This re-verifies the preserved snapshots and operational identity, release, runtime, and authorization state rather than trusting the containment record alone. The output deliberately records `trust_restored: false`: containment recovery is sufficient to enter eradication, not to resume normal production.

5. Production reality

Best Practice: preserve volatile and decision-relevant evidence before mutation, label facts and inference separately, and choose the narrowest containment that closes attacker authority without hiding material business risk.

Production Practice: use synchronized clocks, immutable or independently protected evidence storage, documented custody, predefined incident roles, identity and release kill paths, workload isolation, safe queue handling, legal escalation criteria, and rehearsed communications. Record why service was stopped, degraded, or retained.

6. What changed

Before	After
A correlated alert implied an incident.	**A testable hypothesis separates facts, inference, and unknowns.**
Evidence existed in mutable operational paths.	**A hashed manifest preserves provenance and detects change.**
Scope was an intuition.	**Identity, artifact, workload, data, and business dimensions are explicit.**
Fast response meant deleting systems.	**Preservation precedes staged authority closure.**
Availability and containment competed implicitly.	**Bounded service and preserved state make the trade-off reviewable.**
Containment sounded like recovery.	**The verification record keeps restored trust explicitly open.**

Durable outputs

Artifact	Location	Keep it because
Investigation timeline and evidence manifest	response/timeline.jsonl, response/evidence-manifest.yaml	They preserve event order, provenance, custody, and integrity checks.
Containment decision	response/containment-plan.yaml, generated build/chapter-14-containment.json	It preserves action order, authority closure, continuity choices, ownership, and escalation boundaries.

What You Learned

Incident response is disciplined uncertainty under time pressure. Evidence must remain attributable and unchanged; claims must distinguish observation from inference; and containment must close known authority while preserving business state and later recovery options.

Prove It

> **Independent Practice — Contain suspected payment-provider misuse**
>
> Design an investigation and containment record for unexpected provider charges while legitimate payment requests remain in flight.

Specify facts, inference, unknowns, volatile evidence, integrity checks, identity and credential containment, payment continuity, reconciliation boundaries, communications, and the evidence required before eradication begins.

Next

Northwind has stopped known attacker actions without declaring the environment trustworthy. Chapter 15 removes persistence, rebuilds from trusted roots, rotates affected authority, and reconciles business state before normal operation resumes.

15. Eradicate Persistence and Restore Trust

Chapter 14 stopped known attacker actions, preserved evidence, and retained business state. It did not make the environment trustworthy. The compromised artifact still identifies the contained deployment, an automation credential remains outside the first revocation scope, a payment token was read by attacker-controlled code, and a registry cache may retain material that can recreate the foothold.

1. Containment succeeded, but trust remains open

Northwind's release path is frozen and `order-worker` is isolated. That prevents immediate reuse of known authority, but it does not answer which trust roots produced the affected system or which descendants must be replaced. Returning traffic because the alert is quiet would confuse inactivity with recovery.

Work from the lab working tree using the How to Use This Book procedure. From the DevSecOps lab root, run:

```
make chapter-14-checkpoint chapter-14-contain chapter-14-recover chapter-15-baseline

chapter 15 baseline: harm contained but trust roots and persistence remain unresolved
```

The first three targets recreate Chapter 14 evidence custody, contained operational state, and the generated verification record; `build/` outputs and `response/evidence-manifest.yaml` are not assumed to exist in a fresh working tree. If the Chapter 13 alert or investigating case is missing, run the Chapter 14 opener sequence first. The baseline then requires `trust_restored: false`, confirms that the persistence unknown is still open, and proves that the retained artifact and automation path exist in operational state.

2. The production model: replace trust, reconcile state, then return traffic

> *Theory — Eradication, trust roots, and bounded recovery*
>
> Recovery replaces every affected root and reachable descendant in the modeled scope, then tests security and business outcomes over time.

Containment closes current authority. Eradication removes mechanisms that could restore that authority. A persistence path can be a credential, cache, controller, desired-state record, secret, startup mechanism, or any other route that recreates attacker capability after the visible workload is removed.

A trust root is an input accepted without being derived again inside the current decision: reviewed source intent, identity policy, builder identity, signing authority, configuration identity, or durable business data. Reusing a suspect root during recovery reproduces uncertainty even when the resulting service appears healthy.

Element	Required recovery question	Failure mode
Trust inventory	Which roots and descendants can recreate authority or state?	Recovery omits a persistence path
Rotation scope	Which credentials, keys, and identities must be invalidated or replaced?	Old authority remains reusable
Clean-room rebuild	Did a trusted source, builder, key, and dependency decision produce a new artifact?	Compromised inputs reproduce the artifact
Cache invalidation	Can any retained layer still serve an invalidated digest?	A controller restores the foothold
Desired/actual agreement	Does deployed state match reviewed intent and the admitted artifact?	Recovery validates the wrong system
State reconciliation	Do queue, database, order, and payment outcomes agree?	Security recovery creates hidden customer harm
Recovery criteria	Which security and service outcomes must hold before traffic returns?	A single green sample becomes false confidence
Heightened monitoring	Do detections remain active across recovery windows?	Recovery hides recurrence

Known-good state is not merely a file copied from backup. It is a chain whose roots are independently justified and whose descendants reconcile to the intended result. A clean-room rebuild therefore binds source revision, dependency resolution, builder, signing key, artifact digest, admission decision, deployment evidence, runtime contract, and desired state.

Credential rotation follows the same rule. Issuing a new credential is insufficient if the old credential remains valid, if descendants still trust its outputs, or if caches retain artifacts it published. Rotation succeeds when old authority fails and replacement authority is narrowly usable.

Restored trust is a bounded evidence claim. Northwind can prove that invalidated roots and modeled descendants were replaced, old credentials and artifacts fail, business state reconciles, and detections remain active. It cannot prove that no unmodeled persistence exists anywhere.

3. Build the trust-restoration contract

> **Practice — Inventory roots and reachable descendants**
>
> Open `recovery/trust-inventory.yaml`. Trace identity, automation, builder, artifact, cache, runtime, and payment trust to every modeled dependent.

The inventory marks the incident artifact invalidated, the first compromised session contained, the release automation, registry cache, and exposed payment credential suspect, and the clean rebuild roots trusted. `scope_limit` makes the epistemic boundary part of the artifact rather than a disclaimer added after the result. A descendant of an invalidated root cannot become trusted merely by changing its status: it must be invalidated, replaced, or name the trusted roots from which it was re-derived.

> **Practice — Order eradication before service restoration**
>
> Open `recovery/eradication-plan.yaml`. Evidence verification and persistence detection precede automation rotation and cache invalidation. Rebuild precedes business reconciliation, and monitored service restoration is last.

Reconciliation remains paused during persistence removal. A controller that continues converging toward stale desired state can undo eradication while responders are still rebuilding.

> **Practice — Bind the clean rebuild to inherited recovery roots**
>
> Open `recovery/rebuild-manifest.yaml` and `recovery/verification.yaml`. The manifest consumes the inherited DevOps recovery, incident, GitOps, and observability contracts.

Run:

```
make chapter-15-checkpoint
```

The checkpoint validates the trust graph, rejects dangling or cyclic dependencies, verifies Chapter 14 evidence custody, checks eradication order, and requires the inherited roots: reviewed intent, artifact identity, configuration identity, durable data, and identity policy.

4. Test the design under failure

Cumulative attack — Retained automation and cache persistence

> **Practice — Attempt recovery through a path containment missed**
>
> Model a still-active release automation credential and a registry cache that can serve the invalidated artifact digest.

Severity: critical; premature reconciliation can recreate the production foothold after responders believe the incident is contained.
Plausible harm: renewed malicious execution, repeated order or payment effects, loss of recovery confidence, and a second service interruption.
Potential blast radius: release automation, registry cache, deployment controller, production workload, in-flight orders, and payment reconciliation.
Bounded by: frozen deployment, paused reconciliation, preserved evidence, deterministic local fixtures, and no external execution.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence, reconciliation, and recovery.

Security questions

- **Asset and harm:** Trusted release authority, production state, order outcomes, and recovery evidence are at risk.
- **Trust and authority:** The old automation credential and every artifact reachable through the suspect cache remain untrusted.
- **Detection after prevention fails:** Cache selection and authorization decisions expose the persistence replay while Chapter 13 detection remains active.
- **Evidence of restored trust:** Old automation is denied, the invalidated digest is not servable, a new source-to-runtime chain agrees, terminal business outcomes reconcile, and two consecutive monitored windows pass.

Diagnosis

Run:

```
make chapter-15-attack
```

The controlled replay finds the invalidated digest still servable through the retained cache while `release-workflow` remains active. Reconciliation stays paused, so the lab proves capability without redeploying the artifact.

Containment

Run:

```
make chapter-15-contain
```

The evaluator re-verifies Chapter 14 custody, revokes the missed automation subject, authorizes its narrow replacement, rotates exposed `payment-v2` to `payment-v3` through Chapter 9's authorization mechanism, purges the invalidated cache entry, and confirms the deployment path remains frozen. The resulting eradication record still states `trust_restored: false`.

Recovery

Run:

```
make chapter-15-recover
make chapter-15-verify-recovery
```

Recovery admits a new artifact produced by the trusted builder and signing key, binds that digest through deployment and runtime state, changes credential discovery from detection-only to prevention, and reconciles terminal order and payment outcomes. Verification rejects an unhealthy window, then requires two cadence-adjacent healthy windows. It proves that old automation and payment authority fail, the old digest is unavailable, known intrusion detection still fires, and every trusted descendant is re-derived from trusted or replaced roots. `trust_restored` is computed from those criteria; only then does verification reactivate the runtime contract, unfreeze deployment, and close the incident.

5. Production reality

Best Practice: rebuild from independently justified roots, invalidate every modeled path to the compromised descendants, and make old-authority failure as important as new-authority success.

Production Practice: use dedicated recovery identities and environments, hardware-backed key rotation, immutable provenance, registry and node cache purge controls, GitOps pause and resume gates, durable queue snapshots, provider reconciliation, synchronized evidence windows, and heightened monitoring. Require named owners to accept residual scope limits before traffic return.

A real recovery window should span enough time and workload diversity to expose recurrence. Two modeled windows teach the gate; production duration must follow service behavior, queue depth, scheduled automation, credential lifetimes, and the attacker's plausible re-entry timing.

6. What changed

Before	After
Known attacker actions were contained.	**Modeled mechanisms that could restore authority are invalidated.**
Trust roots had mixed and implicit status.	**Roots, descendants, owners, and scope limits are explicit.**
The compromised digest remained in the operational chain.	**A new digest binds trusted source, build, admission, deployment, and runtime state.**
Automation rotation meant issuing a replacement.	**The old subject is denied and the replacement is narrowly authorized.**
Business state was preserved but not fully recovered.	**Terminal order and payment outcomes reconcile across recovery windows.**
Trust restoration was deliberately false.	**Trust is restored within the declared inventory and evidence limits.**

Durable outputs

Artifact	Location	Keep it because
Eradication and rebuild contract	recovery/eradication-plan.yaml, recovery/rebuild-manifest.yaml	They preserve invalidation order, trusted roots, rotation scope, and the rebuilt chain.
Verified recovery report	build/chapter-15-recovery-verification.json	It separates mechanism, decision, outcome, and recovery evidence while retaining explicit limitations. Regenerate it from make chapter-15-verify-recovery; it is not a substitute for the committed eradication and rebuild contract.

What You Learned

Recovery is not the reversal of containment. It is a new trust decision supported by replaced roots, rejected old authority, reconciled business state, active detection, and sustained evidence. The durable outputs are the eradication/rebuild contract and the bounded recovery-verification report.

Prove It

> **Independent Practice — Recover from a compromised deployment controller**
>
> Design a trust inventory and recovery sequence when the controller credential and desired-state store are both suspect but the production database remains authoritative.

Specify the roots to replace, descendants to reconcile, cache and controller invalidation, clean rebuild chain, business-state gates, consecutive evidence windows, and limitations on the restored-trust claim.

Next

Northwind can now restore trustworthy operation after one modeled compromise. Chapter 16 turns the incident’s control, exception, detection, containment, eradication, and recovery evidence into sustainable governance without replacing risk reasoning with compliance ceremony.

16. Turn Operational Evidence into Sustainable Governance

Chapter 15 restored trust within a declared scope. Northwind now has prevention, detection, containment, eradication, recovery, and business-outcome evidence. Those records still do not create governance by themselves. Without owned objectives, evidence links, review triggers, and improvement work, yesterday's successful controls can become today's green ceremony.

1. Recovery evidence exists, but assurance can still drift

Point-in-time reviews reward collected screenshots and passing reports. Production changes between reviews: exceptions age, telemetry disappears, attack paths gain steps, owners move, and evidence no longer proves the claim it once supported.

Work from the lab working tree using the How to Use This Book procedure after completing Chapter 15. From the DevSecOps lab root, run:

```
make chapter-15-verify-recovery chapter-16-baseline

chapter 16 baseline: checklist governance accepted a false-green assurance report
```

The baseline reads `build/chapter-15-recovery-verification.json` and requires `trust_restored: true`. `chapter-15-verify-recovery` writes that file. It then compares two approaches. A permissive checklist grades the report by its own booleans, so an all-true export passes and a `false` criterion is the only thing that fails. The live assurance evaluator recomputes those criteria and exposes expired exception state, missing telemetry, stale attack-path coverage, incomplete review, and any registered risk the catalog quietly omitted.

2. The production model: assurance is continuous re-proof

> *Theory — Owned objectives, independent evidence, and material change*
>
> Sustainable governance connects risk-backed objectives to implementations, independent evidence, limitations, owners, and review triggers.

Governance decides how security choices remain owned and reviewable over time. Compliance obligations may inform those choices, but a mapped obligation is not evidence that a control works. **GRC (Governance, Risk, and Compliance)** processes become security theater when report completion replaces threat and operational reasoning.

A control objective states the security outcome. A control implementation is the mechanism intended to produce it. Evidence is an attributable record used to evaluate the result. Assurance is the bounded conclusion drawn from current evidence. Keeping those concepts separate prevents a control from grading its own output.

Element	Required governance question	Failure mode
Objective	Which risk-backed outcome must remain true?	Activity replaces security purpose
Implementation	Where and how is the objective enforced?	Intent is mistaken for a control
Ownership	Who can change, review, and repair the control?	Findings remain unowned
Independent evidence	What record outside the claim supports effectiveness?	The report proves itself
Exception state	Is every deviation owned, bounded, compensated, and unexpired?	Temporary bypass becomes permanent
Limitation	What does the evidence not establish?	Assurance expands beyond its scope
Review cadence	When is routine re-evaluation due?	Evidence silently ages
Material change	Which threat, system, owner, or evidence change reopens review now?	The calendar delays a known risk
Improvement	What redesign follows failed assurance?	Teams repair wording instead of controls

Continuous monitoring is not continuous assurance by itself. Telemetry can be complete while a control objective is wrong, and a correct objective can lack current evidence. Assurance combines semantic checks across controls, evidence, exceptions, review state, and system change.

The evidence taxonomy remains explicit:

- **Mechanism evidence** proves that a configured control or evaluator operated.
- **Decision evidence** proves that an owner evaluated required context and made a bounded decision.
- **Outcome evidence** proves that the protected business or security outcome remains healthy.
- **Recovery evidence** proves that harmful authority, persistence, and invalid trust were removed or replaced within the declared scope.

No category substitutes for another. A policy decision cannot prove customer outcomes, and a healthy service cannot prove that old attacker authority fails.

Material change overrides calendar cadence. An incident closure, changed attack path, new cache layer, telemetry contract change, expired exception, or ownership transfer can invalidate assurance immediately. The correct response is to reopen the affected risk decision, not preserve a green status until the next quarterly meeting.

3. Build the control-and-evidence graph

> **Practice — Map objectives to owned implementations**
>
> Open `governance/control-catalog.yaml`. Each objective links a risk and attack path to a named control, owner, implementation, review trigger, and limitation.

The catalog must cover every registered risk and attack path, including support-data purpose as well as the cumulative payment path. Each priority-risk objective also carries a control type so prevention, detection, response, and recovery cannot silently drop out. Registry-cache invalidation explicitly does not cover a node-local cache: that is a different persistence layer, not a naming variant of the same control.

> **Practice — Link independent evidence**
>
> Open `governance/evidence-map.yaml`. Every link names its objective, evidence category, producer, and resolvable `path#fragment`.

The resolver rejects missing fragments in both the evidence map and the report's own `evidence` block, and it prevents the assurance report or challenge fixture from serving as its own evidence. Historical alerts remain useful only because live detection replay is evaluated separately.

> **Practice — Define cadence and material-change triggers**
>
> Open `governance/review-calendar.yaml`. Routine cadence, evidence freshness, and owed obligations are checked against the claim time. Material-change triggers reopen only the objectives they list; a later unrelated review does not clear a different path.

`governance/assurance-report.yaml` records the resulting claim and explicit limits. It does not certify a legal or industry framework.

Run:

```
make chapter-16-checkpoint
```

The checkpoint resolves implementations and evidence, verifies owners, limitations, and risk-register coverage, evaluates live exception expiry, replays detection, checks Chapter 14 custody, validates Chapter 15 recovery, and confirms cadence, freshness, and scoped material-change reviews against the claim time.

4. Test the decision under failure

Independent control failure — The green report that outlived its evidence

> **Practice — Recompute assurance after organizational drift**
>
> Evaluate a report that remains green despite an expired exception claim, an incomplete telemetry window, and a newly observed node-cache redeploy path.

Severity: high; the report can authorize continued reliance on controls whose evidence and threat coverage are no longer current.
Plausible harm: recurrence goes undetected, temporary exceptions persist, risk owners receive false confidence, and investment targets report repair instead of control redesign.
Potential blast radius: supply-chain admission, runtime detection, recovery assurance, risk decisions, audit consumers, and production release governance.
Bounded by: deterministic local fixtures, read-only challenge overlays, explicit findings, and generated correction records.
Primary principles: explicit contracts, trustworthy evidence, reconciliation, recovery, and blast-radius control.

Security questions

- **Asset and harm:** Decision integrity and the controls protecting release and payment outcomes are at risk.
- **Trust and authority:** Report authors may state a result, but only owned evaluators and independent evidence support authority to rely on it.
- **Detection after prevention fails:** Telemetry completeness and attack-path coverage are themselves assurance criteria; a historical alert cannot hide a current gap.
- **Evidence of restored trust:** Chapter 15 recovery remains valid within its limits, but governance assurance fails until the newly identified organizational gaps are corrected.

Diagnosis

Run:

```
make chapter-16-challenge
```

The evaluator rejects the report's `status: pass`. It identifies the expired exception claim, missing deployment and artifact context, a node-cache persistence layer that registry-cache invalidation does not cover, a material-change review that did not include the affected payment objectives, and the wrong report owner.

Correction

The challenge writes `build/chapter-16-assurance-failure.json` instead of editing the report green. The reopened-risk record and improvement backlog are derived from those findings, not copied as constants. The corrected assurance report stays `fail` while telemetry, threat coverage, and review work remain open. Generated correction records live under `build/` and must be regenerated from the evaluator; they are not a substitute for the committed catalog.

This is the governance control loop: fail assurance, reopen risk, restore evidence, redesign controls, and only then recompute the claim. Honest failure is a successful governance outcome.

5. Production reality

Best Practice: treat every assurance claim as a reproducible query over owned controls, current independent evidence, live exception state, explicit limitations, and material-change history.

Production Practice: integrate source control, policy decisions, telemetry quality, incident records, recovery evidence, exception workflows, ownership directories, retention policies, and review scheduling into the organization's assurance process. Preserve immutable report inputs and evaluator versions so conclusions can be reproduced.

Real organizations may map these records to legal, contractual, or industry obligations. That mapping requires qualified interpretation and jurisdiction-specific advice. This chapter provides an engineering evidence model, not certification or legal conclusions.

Automation should make stale evidence visible, not silently renew it. Human review remains necessary for changing threats, residual risk acceptance, ambiguous business impact, and control redesign.

6. What changed

Before	After
Controls worked for one cumulative incident.	**Risk-backed objectives map to owned implementations.**
Evidence existed across chapter outputs.	**Resolvable links classify mechanism, decision, outcome, and recovery evidence.**
Review followed a calendar.	**Material change can reopen assurance immediately.**
Exceptions appeared as report fields.	**Live scope, ownership, compensation, and expiry determine validity.**
A green report was accepted as effectiveness.	**Criteria are recomputed from current operational state.**
Findings ended in report correction.	**Failed assurance reopens risk and creates owned redesign work.**

Durable outputs

Artifact	Location	Keep it because
Control-and-evidence catalog	governance/control-catalog.yaml, governance/evidence-map.yaml, governance/review-calendar.yaml	It preserves objective, owner, implementation, evidence, limitation, and review relationships.
Security improvement backlog	generated build/chapter-16-improvement-backlog.json	It turns the current findings into owned evidence restoration and control redesign. Regenerate it; do not treat the gitignored file as the source of truth.

What You Learned

Sustainable governance is continuous re-proof, not the storage of green reports. The durable skills are maintaining a control-and-evidence catalog and converting assurance failures into an owned security improvement backlog.

Prove It

> **Independent Practice — Govern a new payment-provider integration**
>
> Add an objective for a second provider whose API, owner, telemetry, exception process, and reconciliation evidence differ from the existing path.

Specify the risk and attack path, implementation, independent evidence categories, limitations, routine cadence, material-change triggers, exception checks, and improvement actions that a failed assurance claim would reopen.

Next

Northwind now connects assets, threats, authority, trust, policy, detection, response, recovery, and governance through reproducible evidence. The conclusion assembles those capabilities into one defensible production security system and states the limits that remain.

17. Conclusion — A Defensible Production Security System

Northwind began with inherited DevOps evidence: identified artifacts, bounded workloads, attributable access, and a reconstructable order path. Those capabilities still left the first security question unanswered.

> What must Northwind protect, from which harms, and who owns the decision?

Answering it changed the rest of the work. Assets and harms made threat paths comparable. Threat paths made risk decisions reviewable. Risk decisions named the authority, supply-chain, secret, data, and runtime controls that later chapters had to enforce. Those controls produced evidence. Evidence made detection, containment, eradication, and recovery falsifiable. Governance then asked whether that evidence still proved anything after the incident closed. Failed assurance reopens the risk decision rather than rewriting the report.

The book's central argument is therefore broader than control installation:

> Production DevSecOps is the design of assets, authority, trust, policy, evidence, and recovery so that security claims remain reviewable under attack and over time.

What Northwind can now do

Four capabilities hold the rest of the work together.

Northwind can keep **authority attributable and bounded**. Assets, harms, invariants, and owners are named. Abuse is modeled across trust boundaries, including the support-data path in the same register as the cumulative payment path. Human and automation access is audience-bound and revocable. Privileged operations are governed without treating break-glass as ordinary authority. Secrets and classified data can be rotated, revoked, and replaced rather than merely listed.

It can bind **a verifiable chain from trusted inputs to a deployed digest**. Source and dependencies are admitted only with reviewable origin and ownership evidence. A build and release chain produces an artifact whose digest later admission and runtime evidence can name. Vulnerabilities are ranked by reachability, exploitability, and harm. Policy sits at named enforcement points, ages exceptions against the claim clock, and keeps compensating controls independent. Runtime confinement treats credential discovery, shell execution, and undeclared egress as distinct outcomes.

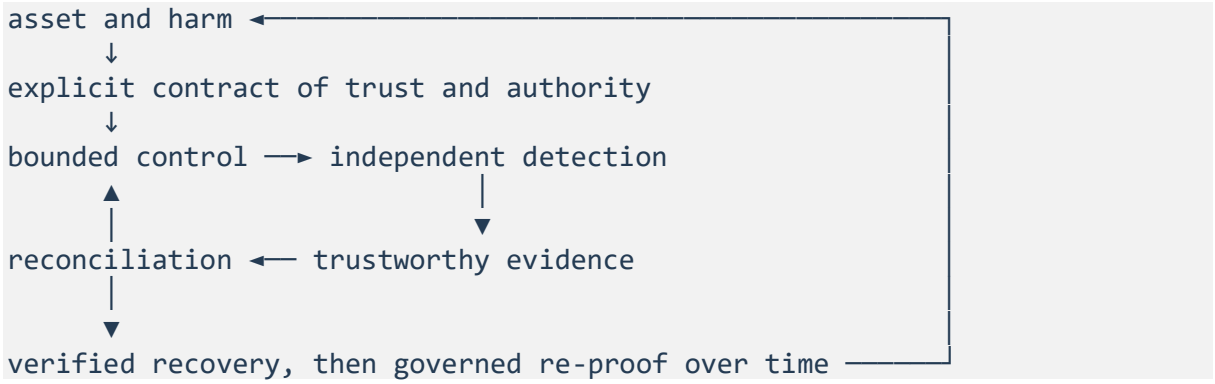
It can produce **evidence that can falsify a claim**. Independently attributable events correlate into detections without mistaking a historical alert for current coverage. A control, report, or recovery record cannot approve itself.

It can **recover and re-prove assurance in ways that are allowed to fail**. Containment preserves evidence, closes known attacker authority, and retains business state without being called recovery. Eradication replaces inventoried persistence paths, rebuilds from trusted roots, and restores trust only within stated evidence limits. Governance maps every registered risk to owned objectives, independent evidence, cadence, and material-change review, then derives an improvement backlog from failed assurance.

Those four capabilities are one system because each creates obligations the others must discharge.

The five principles as one control loop

The series principles now connect as an outer cycle, not a pipeline that ends at recovery:



Failed assurance returns to asset and harm: it reopens the Chapter 3 risk decision and generates redesign work against the controls that were supposed to treat it.

Blast-radius control limits how much identity, artifact, data, and runtime remain exposed while evidence is incomplete.

Explicit contracts make assets, authority, policy, exceptions, outcomes, and failure behavior reviewable.

Trustworthy evidence separates observation from expectation so a control, report, or recovery record cannot approve itself.

Reconciliation turns disagreement among desired, recorded, external, and actual state—including order and payment effects—into an owned transition.

Recovery requires proof that protected outcomes and required trust are healthy again, not merely that an operator completed an action. Chapter 12 classified credential discovery as `detect` rather than `prevent` so evidence of the read would survive. That choice made the payment token a recovery obligation: Chapter 15 had to retire `payment-v2`, issue `payment-v3` through Chapter 9's rotation machinery, and prove that the old version denies while the replacement allows. A detection-over-prevention decision three chapters earlier is why restored trust cannot be a restart.

The four security questions keep that loop honest:

1. What asset and harm drive this decision?
2. What trust and authority are granted?
3. What independent detection remains if prevention fails?
4. What evidence proves trust was restored?

A green service, a green scanner, or a green assurance report answers none of those questions by itself.

What this book does not claim

The companion lab is intentionally deterministic. It proves decision logic, policy evaluation, evidence integrity, correlation, containment order, trust-graph reconciliation, and assurance failure without requiring a live identity provider, registry, Kubernetes control plane, telemetry backend, payment provider, or audit platform.

It does not prove that a particular organization's branch protection, **OIDC (OpenID Connect)** claims, admission controllers, runtime sensors, log retention, secret store, or exception workflow behaves as the fixture does. Production adoption requires replacing each simulated boundary with observed evidence from the real implementation.

The restored-trust claim is bounded on purpose. Chapter 15 can prove that inventoried roots and modeled descendants were replaced, old authority fails, detections remain active, and business state reconciles across consecutive windows. It cannot universally prove that no unmodeled persistence exists. Chapter 16 can fail a false-green report and reopen the affected risk. It cannot certify Northwind against a legal or industry framework.

The incident arc in Chapters 12 through 15 exercises the payment path. The support-data path stays in the same threat, risk, and governance registers, but this book does not produce detection, containment, eradication, or recovery evidence for support-data access.

The scope is also deliberately bounded:

- The DevOps book owns delivery-path evidence, progressive release, data-change compatibility, GitOps operation, and reconstruction from durable operational evidence.
- The Platform Engineering book turns repeated delivery and security capabilities into owned products, paved roads, tenant boundaries, and fleet lifecycle.
- The **SRE (Site Reliability Engineering)** book owns service-level objective programs, error-budget governance, on-call systems, regional-loss architecture, recurring game days, and reliability learning across a service portfolio.

This book extends those foundations with adversarial models, security policy, compromise recovery, and operational evidence for governance. It does not replace a secure-coding textbook, penetration-testing manual, cryptography text, forensics handbook, privacy-law guide, or enterprise architecture reference.

What to carry into production

The portable system is a small set of artifacts, not the lab's YAML, schemas, or checkpoint commands.

Carry the attack-path register and owned risk decisions. They name the assets, harms, treatments, residual risk, and review triggers that later controls must serve. Carry the policy bundle with its enforcement-point map, so authority, admission, exception aging, and compensating controls remain reviewable. Carry the detection hypotheses, so independent observation is not confused with a historical alert. Carry the trust inventory and eradication contract, so recovery replaces modeled roots and descendants instead of restarting a service. Carry the control-and-evidence catalog, so assurance stays tied to owners, limitations, cadence, and an improvement backlog derived from failure.

Treat the rest as scaffolding: fixture subjects, deterministic digests, simulated identity and registry boundaries, generated `build/` reports, and the checkpoint commands that make those fixtures fail on purpose. Reproduce the decisions those artifacts encode; do not copy the files that encoded them here.

The production question to keep asking

When evaluating a new control, tool, exception, or assurance claim, ask:

> What asset and harm justify this decision, what trust and authority cross the boundary, what evidence could falsify the claim, what happens when prevention fails, and how will we prove that trustworthy operation—not merely availability—has been restored?

That is the same reading contract How to Use This Book set. It keeps concepts subordinate to production work. It also prevents “best practice” from becoming a copied control, and “compliance” from becoming a report that outlives its evidence.

The Northwind model and companion implementation are complete for the promise of this book. The lasting skill is not reproducing Northwind's YAML, schemas, or checkpoint commands. It is being able to redesign the same security path when the assets, attackers, identities, dependencies, evidence quality, and recovery obligations are different.

Glossary and Abbreviations

Abbreviations

Abbreviation and full form	Use in this book
API (Application Programming Interface)	A service or provider boundary invoked by software.
CD (Continuous Delivery)	Keeping a change deployable through an automated, controlled path.
CI (Continuous Integration)	Producing fast, trustworthy evidence about proposed source changes.
CISA (Cybersecurity and Infrastructure Security Agency)	The U.S. agency that maintains the known-exploited vulnerability catalog cited as exploitation intelligence.
CVSS (Common Vulnerability Scoring System)	A severity scoring framework; its Base score is not by itself a risk decision.
GRC (Governance, Risk, and Compliance)	Organizational processes for owned risk, control evidence, and obligations; a mapped obligation is not proof that a control works.
IMDSv2 (Instance Metadata Service version 2)	AWS instance-metadata access that requires a session-oriented request rather than an unauthenticated GET.
KEV (Known Exploited Vulnerabilities)	CISA's catalog of vulnerabilities with evidence of exploitation in the wild.
NIST (National Institute of Standards and Technology)	The U.S. standards body whose incident-response publication is cited for current risk-management framing.
OIDC (OpenID Connect)	Providing verifiable identity claims used by hosted builds or workload federation.
SBOM (Software Bill of Materials)	A structured inventory of components associated with an artifact.
SLSA (Supply-chain Levels for Software Artifacts)	A framework for reasoning about build provenance and platform assurance.
SRE (Site Reliability Engineering)	Reliability engineering through service objectives, operations, and learning.
STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)	A mnemonic threat catalog used as a prompt, not as proof of coverage.

Production terms

Asset
Something Northwind values because its compromise can cause a meaningful business, customer, security, or operational harm. Identity and authority can be assets; a credential is replaceable material that represents them.

Assurance
A bounded conclusion drawn from current independent evidence that a control objective still holds. A green report is not assurance by itself.

Attack path
Connected actions across real flows and trust boundaries that can progress a threat actor toward a named harm.

Attestation
A signed statement about a subject. A valid signature authenticates the signer and unchanged claims; it does not automatically make the signer, builder, inputs, or target trusted.

Blast radius
The identities, artifacts, data, environments, workloads, or authority exposed to a change, attack, or failure.

Break-glass access
A predesigned emergency path that grants exceptional authority. It remains attributable, independently approved, time-bound, audited, and removed after review.

Containment
Closing identified attacker access and execution paths while preserving evidence and required business state. Containment is not eradication or restored trust.

Decision evidence
Proof that an owner evaluated required context and made a bounded decision, including uncertainty, rationale, owner, and review trigger.

Detection hypothesis
A statement of which attack behavior should produce which evidence, within what window, with which context, and what response follows.

Enforcement point
A boundary with the context and authority to allow, deny, transform, or observe a transition according to a named policy rule.

Eradication
Removing modeled mechanisms that could restore attacker authority after containment, including credentials, caches, controllers, secrets, and other persistence paths.

Exception
A bounded policy decision with owner, rationale, narrow scope, compensating controls, evidence, effective time, expiry, and removal path. It is not the absence of policy.

Finding
A tool's claim that a weakness or vulnerability may exist. Findings inform risk decisions; they are not risk decisions.

Harm
The adverse outcome if a security invariant is violated.

Hermeticity
The property that declared inputs determine a build without undeclared network or host dependencies. Isolation bounds interference between builds; neither property implies the other.

Inherent risk
The assessed risk of a path before the selected control portfolio.

Mechanism evidence
Proof that a configured control or evaluator operated: a denial, admission, alert, hash, or other observation of the mechanism itself.

Outcome evidence
Proof that the protected business or security outcome remains healthy, such as correct terminal order state or failed unauthorized payment effects.

Persistence path
A credential, cache, controller, desired-state record, secret, startup mechanism, or other route that can recreate attacker capability after the visible workload is removed.

Provenance
Evidence binding an artifact subject to build facts such as source, revision, builder, parameters, and build type.

Reachability
Time-bound evidence that a particular version, configuration, and path can exercise a finding. Unreachable does not mean permanently harmless.

Reconciliation
Observing disagreement among desired, recorded, external, and actual state—including order and payment effects—and applying or proposing a controlled transition.

Recovery
Verified restoration of protected outcomes and required trust, not merely completion of a corrective action.

Recovery evidence
Proof that harmful authority, persistence, and invalid trust were removed or replaced within the declared scope, that detections remain active, and that business outcomes recover.

Red capability
A runnable checkpoint result proving that a production capability is absent or unsafe.

Residual risk
What remains after existing and planned controls operate as expected. Residual risk is never automatically zero.

Restored trust
A bounded evidence claim that inventoried roots and modeled descendants were replaced, old authority fails, trusted state is reconciled, detections remain active, and business outcomes recover. It is not a universal proof that no unmodeled persistence exists.

Root of trust
The identities, keys, policy sources, and verification mechanisms whose correctness a trust decision depends on. Adding signatures expands the evidence graph; it does not eliminate roots.

Secret
Material whose possession grants capability. An identity is an attributable subject with claims evaluated in context; possession of a secret often weakens that attribution.

Security invariant
A required or prohibited outcome that should remain true across normal operation, change, attack, failure, containment, and recovery. Later controls, detections, and recovery evidence must preserve it.

Threat
A circumstance or actor capability that could violate a security invariant.

Trust boundary
A transition where the receiving side must decide whether to accept claims, data, identity, intent, or authority from a different trust context. It is not merely a network segment.

Trust root
An input accepted without being derived again inside the current decision, such as reviewed source intent, identity policy, builder identity, signing authority, configuration identity, or durable business data.

Vulnerability
A weakness that can be exercised under relevant conditions to violate a security property.

Workload contract
Declared identity, artifact, process, privilege, filesystem, and network behavior expected for one workload.

Workload identity
An attributable runtime subject exchanged for short-lived, audience-bound dependency access without distributing a reusable credential where federation is supported.

References

The chapter text cites sources at the decision they support. This consolidated list is organized by chapter for review and maintenance. Product behavior and documentation change; verify version-specific details before applying an example to production.

Chapters that make no product or standards claim—asset modeling, qualitative risk, privileged delegation, secrets lifecycle, data classification, governance, and the conclusion—have no external entries. Their decisions are Northwind's, not a cited framework's.

Chapter 2 — Threats across trust boundaries

- **STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)** threat categories

Chapter 4 — Attributable access

- **OIDC (OpenID Connect)** Core

Chapter 6 — Source and dependency trust

- Dependency confusion
- GitHub protected branches

Chapter 7 — Verifiable build and release

- **SLSA (Supply-chain Levels for Software Artifacts)** build requirements
- SLSA artifact verification guidance

Chapter 8 — Vulnerability prioritization

- **CVSS (Common Vulnerability Scoring System)** v4.0 specification
- **CISA (Cybersecurity and Infrastructure Security Agency) KEV (Known Exploited Vulnerabilities)** catalog

Chapter 11 — Policy enforcement

- Kubernetes admission controllers

Chapter 12 — Constrain workloads and detect runtime abuse

- Kubernetes security context
- AWS instance metadata and **IMDSv2 (Instance Metadata Service version 2)**

Chapter 13 — Security evidence and detections

- OpenTelemetry signals
- OpenTelemetry context propagation

Chapter 14 — Investigation and containment

- **NIST (National Institute of Standards and Technology)** SP 800-61r3